SE491 Project
Software Engineering Department
Tuesday, 17 Dec 2025

SE491
Capstone Project

Nubl: A Community-Centered Digital Aid and Food Support Ecosystem Platform

Prepared By:

Salem Hussameldeen Shurrab (4211168)

Bashar Mustafe Naial (4310757)

Osamah Khaled Al-Suraimi (4211315)

Supervision By:

Dr. Basit Shahzad
Software Engineering Associate Professor

I. Anti-Plagiarism Declaration

This is to declare that the above publication produced under the supervision of Dr. Basit Shahzad, having the title "Nubl Platform" is the sole contribution of the authors and no part hereof has been reproduced illegally, which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Date: 31 Dec 2025
Author(s):
Name: **Salem Hussameldeen Shurrab** Signature: ***Salem Shurrab***
Name: **Bashar Mustafe Naial** Signature: ***Bashar Naial***
Name: **Osamah Khaled Al-Suraimi** Signature: ***Osamah Al-Suraimi***

II. Acknowledgment

We would like to express our special thanks to Dr. Basit Shahzad who helped and guided us to learn new things. Furthermore, many thanks to the Software Engineering Department and College of Computer Science as well for their support.

Throughout Capstone I, we transformed the NUBL concept from an initial idea into a structured software engineering project by analyzing real-world challenges related to food assistance, privacy, and transparency. These challenges were translated into clear system goals, functional requirements, and non-functional requirements that guided the overall project direction.

We applied core software engineering principles to design realistic system models, including use cases, activity, components, and state diagrams. Engagement with real stakeholders and charitable organizations helped validate the practicality of the concept and refine the project scope, establishing a solid technical and conceptual foundation for Capstone II and strengthening our ability to design secure, privacy-aware systems with social impact.

III. Abstract

Nubl is a web-based, neighborhood-level charitable platform designed to modernize food assistance while preserving privacy, dignity, and accountability. The project is motivated by the concept of organized digital charity (ṣadaqah), aiming to enable individuals to support essential grocery and meal needs within their local communities through a transparent and ethical workflow.

Traditional food assistance programs often depend on manual processes, physical vouchers, or fragmented coordination between donors and providers. These approaches lead to delays, limited visibility into fund usage, administrative overhead, and, in many cases, public exposure of beneficiaries' identities. Such challenges reduce trust, discourage participation, and compromise dignity. Nubl addresses these issues by introducing a unified digital system that automates verification, request handling, and fulfillment while minimizing personal data exposure.

The proposed solution connects three main user groups within a single city: Gracious Neighbors who contribute funds, Nubl Neighbors who request essential groceries or meals, and Nubl Bridges (local supermarkets or restaurants) who fulfill approved requests. Donations are processed securely through the MyFatoorah payment gateway and pooled into a general city fund. Neighbors submit requests within a fair, per-person allowance, with exceptional cases flagged for administrative review. Approved requests are fulfilled using secure, one-time QR codes, and providers upload proof of delivery. Every step of the process is recorded in an audit log to ensure transparency and traceability without revealing personally identifiable information.

This project follows a Waterfall-based methodology, focusing in Capstone I on requirements analysis, system design, and validation of the Minimum Viable Product (MVP) for a single-city pilot. The outcome of this phase is a well-defined, privacy-first MVP that demonstrates how digital tools can support fair, transparent, and dignity-preserving charitable food assistance. Future work in Capstone II will focus on implementation, pilot deployment, and system expansion.

Keywords	***Description***
NUBL Platform	A digital humanitarian platform, connecting Gracious Neighbors, NUBL Bridges, and NUBL Neighbors to deliver essential grocery needs or meals with privacy, dignity, and full transparency.
Gracious Neighbor (Donor)	A donor who contributes funds through online or bank payments to support recipients in covering essential needs.
NUBL Neighbor (Recipient)	A person in need who requests essential grocery items or meals from approved partner providers via the platform.
NUBL Bridge (Provider)	A registered supermarket, grocery store, or restaurant that fulfills approved requests and receives payments from the city support fund.
Request	The digital form submitted by a NUBL Neighbor to request specific essential grocery items or meals from a provider.
QR Code	A secure, one-time digital code used by NUBL Bridges to verify and redeem approved requests instantly and safely through the platform.
City Support Fund	A pooled, city-level account that gathers contributions from Gracious Neighbors (via online or bank transfers) and is used to pay registered NUBL Bridges for completed requests.
Allowance	A predefined limit that determines how much assistance a NUBL Neighbor can receive (based on value or frequency), ensuring fairness and balance.
Privacy-by-Design	A foundational principle ensuring that privacy, minimal data collection, and user dignity are integrated at every stage of the NUBL Platform's architecture.
KYC (Know Your Customer)	The process of verifying a user's identity to ensure they are legitimate.
PII (Personally Identifiable Information)	Any personal data that can identify an individual.

Table 1: Keywords and Descriptions

IV. ***Table of Contents:***

I. Anti-Plagiarism Declaration 2
II. Acknowledgment 3
III. Abstract 4
IV. *Table of Contents:* 1
V. List of Tables 1
VI. List of Figures 2
.I Acronym 3
1. Introduction and Background 5
1.1. Purpose of the Project 5
1.2. Problem Statement 5
1.3. Objectives (MVP) 5
1.4. Project Scope 6
1.5. Constraints and Assumptions 6
1.6. Proposed Solution 8
2. Literature Review 9
2.1 Literature Review Methodology (Kitchenham SLR) 9
2.2 Digital Charity Resilience and Trust 9
2.3 Digital Payments for Humanitarian and Food Assistance 9
2.4 Privacy-by-Design and Stakeholder Considerations in Digital Social Systems 10
2.5 Blockchain and Digital Trust for Food Systems and Traceability 11
2.6 Existing Digital Aid Platforms and Their Limitations 12
2.7 Synthesis and Research Gap 13
3. Methodology 14
3.1. Overview 14
3.2. Methodologies To Consider 14
3.3. Our Methodology 16
3.4. Project Plan 19
4. System Analysis 22
4.1 Overview 22
4.2 System Requirement Sources 23
4.3 Functional Requirements (FRs) 23
4.4 Non-Functional Requirements (NFRs) 34
4.5 Use Case Diagram 36

4.6 System Activity Flow Diagram 37
5. System Design 38
5.1 Overview 38
5.2 Candidate Architectures 38
5.3. Selected System Architecture 45
5.3.1 Architecture Overview 45
5.3.2 Justification for the Selected Architecture 48
5.4. A More Detailed Design 49
5.4.1. Entity Relation Diagram - ERD 49
5.4.2. Class Diagram 50
5.4.3. Sequence Diagram- User Registration & Approval 51
5.4.4. Sequence Diagram- Beneficiary Request Creation & Allowance Check 52
5.4.5. Sequence Diagram - Admin configures system-wide allowance (FR-17.1) 53
5.4.6. Activity Diagram- Beneficiary Request & Fulfillment 54
5.4.7. State Diagram - Request Lifecycle 55
5.4.8. Sequence Diagram – Payment 56
5.4.9. Activity Diagram – Payment 57
5.5.1. State Diagram – Payment Process State 58
5.5.2. Sequence Diagram – Donor Impact Dashboard 59
5.5.3. Activity Diagram – User Registration & Approval 60
5.5.4. Sequence Diagram – QR Redemption and Atomic Balance Update 61
5.5.5. Activity Diagram – QR Redemption and Atomic Balance Update 62
5.5.6. State Diagram – QR Redemption and Atomic Balance Update 63
5.5.6. Component Diagram 64
6. Future work (To be Done in Capstone 2) 65
7. KPI 4.1 – Professional and Ethical Standards 65
7.1 Ethical Issues and Professional Code of Conduct 65
7.2 Ethical Case Studies 66
7.3 Personal Ethical Reflection 66
7.4 Regulatory Laws and Ethical Guidelines 66
8. KPI 4.2 – Global, Economic, Environmental and Societal Impact 66
8.1 Global markets and communities impact 66
8.2 Economic benefits and costs - high-level assessment 66

8.3 Environmental impact 67
9. Conclusion 67
10. References 69
Appendices 71
Appendix A: Teamwork and Project Management Evidence 71
Appendix B: Proof of Concept Evidence 73
Appendix C: Continuous Development 76
Appendix D: Logs Table 77

V. List of Tables

Table 1: Keywords and Descriptions 5
Table 2 :Acronyms and Descriptions 4
Table 3: Phase-Specific Constraints (Capstone I) 7
Table 4: Overall Project Constraints 8
Table 5: Project Plan 19
Table 6 : System Requirement Sources 23
Table 7 : Functional Requirements (FRs) 34
Table 8 : Non-Functional Requirements (NFRs) 35
Table 9 : Candidate Architectures Final Ranking 45
Table 10 : Economic Benefits and Costs Assessment 67
Table 11: Supervisor Meetings 72
Table 12: Stakeholder Meetings 72
Table 13: Team Sync Meetings 72
Table 14: Jira Task Tracking Evidence 73
Table 15: Risk Register 73
Table 16: What We Learned 77
Table 17: Logs Table 78

VI. List of Figures

Figure 1: Full WBS Picture (3.4.1) 20
Figure 2: Work Breakdown Structure For Milestone 1 (3.4.1.1) 20
Figure 3: Work Break Down Structure For Milestone 2 (3.4.1.2) 21
Figure 4: Work Break Down Structure For Milestone 3 (3.4.1.3) 21
Figure 5: Use Case Diagram (4.5) 36
Figure 6: System Activity Flow Diagram (4.6) 37
Figure 7: Architecture A (Simple AWS Cloud Architecture) (5.2.1) 38
Figure 8: Architecture B (AWS Cloud Architecture) (5.2.2s) 40
Figure 9: Architecture C (Architecture 3-Tier Web Design) (5.2.3) 42
Figure 10: Our System Architecture (5.3) 45
Figure 11: Entity Relation Diagram – ERD (5.4.1) 49
Figure 12: Class Diagram (5.4.2) 50
Figure 13: Sequence Diagram – User Registration & Approval (5.4.3) 51
Figure 14: Sequence Diagram – Beneficiary Request Creation & Allowance Check (5.4.4) 52
Figure 15: Sequence Diagram - Admin configures system-wide allowance (5.4.5) 53
Figure 16: Activity Diagram- Beneficiary Request & Fulfillment (5.4.6) 54
Figure 17: Sequence Diagram – Payment (5.4.8) 56
Figure 18: Activity Diagram – Payment (5.4.9) 57
Figure 19: State Diagram – Payment Process State (5.5.1) 58
Figure 20: Sequence Diagram – Donor Impact Dashboard (5.5.2) 59
Figure 21: Activity Diagram – User Registration & Approval (5.5.3) 60
Figure 22: Sequence Diagram – QR Redemption and Atomic Balance Update (5.5.4) 61
Figure 23: Activity Diagram – QR Redemption and Atomic Balance Update (5.5.5) 62
Figure 24: State Diagram – QR Redemption and Atomic Balance Update (5.5.6) 63
Figure 25 Jira Tasks 73
Figure 26: UI UX Login / role access 74
Figure 27: UI UX Beneficiary request creation 74
Figure 28: UI UX Admin approval screen 75
Figure 29: UI UX Provider QR redemption + receipt upload 75
Figure 30: UI UX Donor dashboard (aggregated) 76

I. Acronym

Acronym	*Description*
KYC (Know Your Customer)	Identity verification to ensure a user is legitimate.
PII (Personally Identifiable Information)	Personal data that can identify an individual.
OTP (One-Time Password)	A temporary code used to verify a user during login/registration.
RBAC (Role-Based Access Control)	Access control based on user roles (Admin/Donor/Beneficiary/Provider).
RLS (Row-Level Security)	Database policy that restricts which rows a user can access.
JWT (JSON Web Token)	Token used for authentication/authorization in web APIs.
API (Application Programming Interface)	A backend interface that clients use to send/receive data.
TTL (Time-To-Live)	Expiration duration for something like a QR token.
TLS (Transport Layer Security)	Encryption for data in transit (for example HTTPS).
AES (Advanced Encryption Standard)	Encryption algorithm for data at rest (for example AES-256).
CSV (Comma-Separated Values)	A simple file format used for exports/audit logs.
SLA (Service Level Agreement)	Target service availability/performance (for example uptime).
DDoS (Distributed Denial of Service)	Attack that floods a service with traffic to make it unavailable.
WAF (Web Application Firewall)	Filters web traffic to block attacks like XSS/SQL injection.
CDN (Content Delivery Network)	Caches and serves static content faster (for example CloudFront).
VPC (Virtual Private Cloud)	Isolated cloud network for secure infrastructure on AWS.
ELB (Elastic Load Balancer)	Distributes incoming traffic across servers/instances.
ASG (Auto Scaling Group)	Automatically increases/decreases compute instances based on load.
S3 (Simple Storage Service)	AWS object storage for files (receipts/audit exports).
RDS (Relational Database Service)	Managed AWS relational database service.
NFR (Non-Functional Requirement)	Quality requirement (performance, security, usability…).
FR (Functional Requirement)	Feature/behavior requirement (what the system must do).
SLR (Systematic Literature Review)	Structured method for reviewing research (Kitchenham style).

GDPR (General Data Protection Regulation)	EU data protection regulation (privacy principles).
PDPL (Personal Data Protection Law)	Saudi personal data protection law referenced in your report.
OWASP (Open Worldwide Application Security Project)	Security guidance (for example Top 10 risks).
NGO (non-governmental organization)	An NGO, or non-governmental organization, is a voluntary group of individuals or organizations that operate independently of any government.

Table 2 :Acronyms and Descriptions

1. Introduction and Background

1.1. Purpose of the Project

The purpose of the Nubl project is to design and develop a web-based platform that simplifies food assistance while protecting user privacy and dignity. The system connects three main user groups: Gracious Neighbors who fund the assistance, Neighbors who request groceries, essential items, and meals, and Providers (supermarkets or restaurants) who fulfill these requests. The platform aims to ensure that every transaction, from funding to fulfillment, is transparent, traceable, and free of personal data exposure. By using digital tools and clear workflows, Nubl seeks to make charitable food distribution faster, fairer, and more efficient.

1.2. Problem Statement

Traditional food assistance programs often rely on manual processes, physical vouchers, or fragmented communication between donors and service providers. This results in delays, inconsistent eligibility checks, and a lack of accountability. In many cases, neighbors must publicly identify themselves to receive help, which can compromise their dignity and privacy.

Gracious Neighbors, on the other hand, have limited visibility into how their funds are used, while providers face administrative burdens when verifying requests. These challenges highlight the need for a unified digital solution that automates verification, tracks funds, and provides clear audit trails without exposing personal identities.

1.3. Objectives (MVP)

The primary purpose of the Nubl Platform is to provide a dignified, privacy-preserving, and transparent digital solution for neighborhood-level food assistance. The system aims to reduce administrative burden, protect beneficiary identity, and increase trust between donors, beneficiaries, and service providers by replacing manual and fragmented aid processes with a unified digital workflow.

To achieve this purpose, the main objectives of this project are to develop a Minimum Viable Product (MVP) that:

1. Enables Gracious Neighbors to securely contribute funds to a centralized city support fund through the MyFatoorah online payment gateway.
2. Allows Nubl Neighbors to request approved essential grocery items or meals within a fair, per-person allowance to ensure equitable distribution of support.
3. Enables Nubl Bridges (providers) to redeem approved requests using a secure, one-time QR code and upload proof of fulfillment (receipt) for accountability.
4. Maintains privacy and transparency by minimizing personal data exposure and recording all key actions through an immutable audit log.
5. Supports administrators in managing requests, exceptions, and policies to ensure fairness, regulatory compliance, and operational control.

By achieving these objectives, the Nubl Platform ensures that food assistance is delivered efficiently, fairly, and respectfully, while building trust among all stakeholders and laying a solid foundation for future expansion in Capstone II.

1.4. Project Scope

This project focuses on designing a Minimum Viable Product (MVP) of the Nubl Platform as a single-neighborhood pilot within one city. The scope is intentionally limited to ensure feasibility within the Capstone I timeframe while covering the core workflow of neighborhood-based food assistance.

1.4.1 In-Scope (Included Features)

The following high-level features are included in the scope of the MVP:

- A responsive web-based application supporting four user roles: Gracious Neighbor (Donor), Nubl Neighbor (Beneficiary), Nubl Bridge (Provider), and Administrator.
- User registration and KYC-Light verification with minimal personal data collection.
- A centralized city-level support fund managed through the MyFatoorah online payment gateway.
- A digital request submission system for essential grocery items or meals, subject to per-user allowance limits.
- An administrative module for managing neighborhoods, user approvals, requests, and exception handling.
- A QR-code–based request redemption mechanism for providers, including receipt upload as proof of fulfillment.
- A transparent audit and reporting mechanism using CSV exports to track requests, approvals, payments, and redemptions.

1.4.2 Out of Scope (Excluded Features)

The following features are explicitly excluded from the scope of this MVP:

- Multi-city or nationwide deployment.
- Integration with payment gateways other than MyFatoorah.
- Advanced automation, AI-based eligibility assessment, or fraud detection.

These excluded features are considered potential extensions and may be addressed in future iterations during Capstone II or later system versions.

1.5. Constraints and Assumptions

In Capstone I, we will focus on an MVP design for the Nubl Platform. Because this is a student project, we had limits on time, resources, and scope. These limits affected our requirements and system design decisions.

1.5.1 Constraints

The Nubl Platform is subject to a set of constraints that influenced both the current Capstone I phase and the overall system design. These constraints were considered to ensure feasibility, compliance, and alignment with academic and real-world conditions.

A. Phase-Specific Constraints (Capstone I)

The following constraints apply specifically to the Capstone I phase, which focuses on analysis and design:

Constraint Type	Constraint	How it Affected Our Work
Budget	Use of free or low-cost tools for hosting, database, and storage.	Encouraged a simple, cost-efficient architecture and avoided expensive enterprise solutions.
Timeline	Capstone I is limited to analysis and design; full implementation is planned for Capstone II.	The MVP scope was restricted to core workflows only.
Technology	The MVP is web-based and supports all roles in a single system.	A unified web platform was designed for Beneficiary, Donor, Provider, and Admin roles.
Payment	Only one payment gateway is used in the MVP.	Donation workflows were designed exclusively around MyFatoorah.
Scope	Features such as multi-city support, and advanced automation are excluded from this phase.	Requirements and diagrams focus strictly on MVP functionality.

Table 3: Phase-Specific Constraints (Capstone I)

B. Overall Project Constraints

The following constraints apply to the Nubl Platform across all phases of the project, including future implementation and deployment:

Constraint Category	Description
Platform Constraint	The system must remain web-first to ensure accessibility across devices without requiring native mobile applications.
Design Constraint	The architecture must support privacy-by-design principles, role separation, and auditability without over-engineering.
Implementation Constraint	The system must be feasible for development by a small academic team using widely available technologies and frameworks.
Security Constraint	All system components must enforce strong access control, encryption for data at rest and in transit, and secure authentication mechanisms.
Regulatory Constraint	The platform must comply with Saudi Personal Data Protection Law (PDPL) and align with general privacy principles such as GDPR.
Operational Constraint	Providers may require onboarding and basic training to use QR redemption and receipt upload workflows effectively.

Data Retention Constraint	Personal and transactional data must be minimized and retained only for the duration required by operational and legal needs.
Environmental Constraint	The system should minimize unnecessary storage and processing to reduce operational and environmental impact.

Table 4: Overall Project Constraints

1.5.2 Assumptions

To keep the MVP clear and achievable, we used the following assumptions:

- The pilot starts in one city with a limited number of users.
- Providers are verified and trained before they start redeeming QR codes.
- Donors are allowed to view statistics only, not beneficiary identity.
- The system uses one currency and one gateway (MyFatoorah) in the MVP.

1.6. Proposed Solution

The proposed solution for Nubl is a web-based platform that implements a secure, end-to-end digital workflow for managing charitable food assistance at a neighborhood level. The system operationalizes the previously defined objectives by orchestrating funding, request handling, approval, and fulfillment through clearly separated roles and automated controls.

Gracious Neighbors contribute funds through the MyFatoorah payment gateway, with all donations pooled into a general city fund. These funds are managed centrally and governed by predefined allowance and policy rules. Nubl Neighbors submit requests for essential grocery items or meals within their assigned allowance, while exceptional cases are automatically flagged for administrative review to ensure fairness and flexibility.

Once the admin approves a request, the system automatically issues a secure one-time QR code that the neighbor presents to the provider. The provider redeems the code, fulfills the request, and uploads a receipt or proof of delivery. Every transaction from request to fulfillment is logged in an audit CSV file to maintain transparency and traceability.

Gracious Neighbor can view aggregated statistics, including total grocery items delivered, number of neighbors served, top providers, and fund utilization. No personally identifiable information (PII) is exposed at any point. To encourage community engagement, Nubl introduces non-intrusive recognition features such as Popular This Week badges while keeping user identities private.

This approach ensures fairness, reduces administrative workload, and provides Gracious Neighbor with verified impact data. The system's modular architecture supports expansion to multiple cities, deeper payment gateway integration, and native mobile applications in Capstone II.

For third-party integrations, the NUBL Platform relies on several external services to ensure secure and efficient operation:

- Online Payment Gateway (MyFatoorah): Handles all fund transactions securely and generates receipts.
- Email Service (Mailgun): Sends account verification, OTPs, and notification emails.
- Analytics Service (Google Analytics): Monitors user behavior and system usage to improve usability and performance.

2. Literature Review

2.1 Literature Review Methodology (Kitchenham SLR)

Following Kitchenham's (2007) guidelines for Systematic Literature Review in Software Engineering, the research team identified and analyzed relevant peer-reviewed studies from IEEE Xplore, ScienceDirect, and Wiley Online Library (2021–2025). Search terms included "digital charity platforms," "privacy-by-design," and "trust in aid technology." Sources were screened for credibility, recency, and relevance to humanitarian software systems.

2.2 Digital Charity Resilience and Trust

Richey, Brooks, and Ravishankar (2024) demonstrate that digital tools play a critical role in enabling charities to sustain operations and deliver social value during crises when systems are designed with resilience in mind. Their study of eBay's Charity Connect initiative highlights how privacy-preserving design, efficient digital processes, and transparent data practices strengthen trust between donors, beneficiaries, and platform operators. This research shows that even resource limited organizations benefit from investing in robust, privacy-aware digital infrastructures. Inspired by these findings, Nubl adopts similar principles privacy-by-design, transparent audits, and localized community engagement to ensure continuity, accountability, and dignity in food assistance delivery.

2.3 Digital Payments for Humanitarian and Food Assistance

Recent empirical work shows that digital payments can successfully deliver humanitarian aid and significantly improve welfare outcomes in fragile settings. Callen et al. (2025) evaluate a large-scale digital cash transfer program for extremely poor, female-headed households in Afghanistan, implemented via a private digital payment platform in partnership with community and nonprofit actors. The study finds substantial improvements in food security and mental well-being, with 99.75% of beneficiaries successfully using the transfers despite very low levels of digital literacy and difficult operating conditions. Importantly, stringent audits detected no evidence of diversion, and delivery costs were under USD 0.07 per dollar delivered about USD 0.10 less than the World Food Programme's global average for cash-based transfers, demonstrating that digital channels can be both cost-effective and secure compared to traditional mechanisms (Callen et al., 2025).

Similar results emerge from Pakistan. Fajardo-Steinhäuser and Sarwar (2025) report on a randomized controlled trial with 2,450 extremely vulnerable women in Sindh, who received four monthly transfers either through digital wallets, through more traditional cash-like methods, or not at all (control). Although 84% had no formal education and only 5% had ever used mobile money, over 99% of digital recipients were able to access their funds, matching cash access rates. The transfers were used primarily for food, medicine, debt repayment, and children's clothing. Digital aid achieved at least comparable and in some indicators better outcomes than cash: food insecurity fell more (about 22% larger reduction than cash), diet diversity improved, and mental well-being increased in both groups. Digital recipients were also significantly more likely to afford medicines when needed and spent less time and money traveling to collect aid, reducing the burden on already fragile households (Fajardo-Steinhäuser & Sarwar, 2025).

Taken together, this evidence suggests that digital aid is not only feasible but highly effective even among low-literacy, vulnerable women in fragile states. It also challenges the common assumption that digital divides make wallet-based aid impossible in humanitarian contexts. With minimal training and appropriate support, recipients can reliably use digital accounts. For our project, this literature supports the idea that a digital wallet or voucher mechanism for food assistance can improve food security, reduce delivery costs, and increase convenience for beneficiaries provided that usability and support are designed carefully.

2.4 Privacy-by-Design and Stakeholder Considerations in Digital Social Systems

While digital aid can be effective, it raises critical questions around privacy, data protection, and trust, especially when sensitive data (identity, location, consumption patterns) are collected and processed. Carboni et al. (2023) propose a privacy-by-design framework for systems in assisted living, personalised care, and wellbeing, with a focus on European Active and Assisted Living / Active Healthy Ageing (AAL/AHA) projects. Their work is highly relevant to any digital system that handles personal data in a social or health context, including digital aid platforms.

Starting from the General Data Protection Regulation (GDPR), particularly Article 25, the authors emphasize that privacy must be embedded by design and by default throughout the system lifecycle, not added as an afterthought. This includes technical and organizational measures such as pseudonymization, data minimization, and ensuring that by default only the minimum necessary personal data are processed, stored for limited periods, and not exposed to unnecessary parties (Carboni et al., 2023). They also distinguish between information security (confidentiality, integrity, availability – the CIA triad) and cybersecurity, and highlight additional privacy requirements such as anonymity, pseudonymity, unlinkability, undetectability, and unobservability.

A key contribution of Carboni et al. (2023) is their stakeholder and dimension mapping. They identify four main stakeholder groups:

- Primary end users (For example older adults / beneficiaries),
- Secondary end users (For example care organizations, providers),
- Solution and platform developers, and
- Authorities and facilitators (For example municipalities, regulators, insurers).

They then map these groups onto two main dimensions: a technical dimension (devices, integration, security, data analytics, visualization) and a contextual dimension (legal or administrative context, ethics and privacy, data governance, IP and business aspects). Their analysis shows that many projects focus firmly on the technical aspects (For example sensor platforms, analytics) but underinvest in the contextual work: Information security awareness (ISA) training, user understanding of how data are used, and clear communication of privacy protections. The authors argue that user acceptance, perceived safety, and involvement (For example through gamification or participatory design) are critical to project success, particularly for populations that are less comfortable with technology.

For our context, this suggests that a digital food assistance platform should be built on privacy-by-design principles, with:

- Clear stakeholder mapping (beneficiaries, donors, providers, municipalities, platform operators),
- Explicit treatment of legal/privacy requirements (For example local equivalents to GDPR, data retention, informed consent), and
- Active efforts to raise security and privacy awareness among both users and staff.

Rather than treating privacy as a purely technical issue (encryption, access control), the literature emphasizes that it is also a social and organizational problem that must be addressed through training, communication, and inclusive design.

2.5 Blockchain and Digital Trust for Food Systems and Traceability

Beyond digital payments and privacy, there is growing interest in how blockchain and distributed ledger technologies (DLT) can improve transparency and trust in food systems. George and Al-Ansari (2023) introduce GM-Ledger, a blockchain-based framework for authenticating certificates in international food trade, using Qatar as a case study. Their work focuses on maritime imports and trade documentation, but the underlying principles are applicable to any food-related system that requires trust, traceability, and fraud resistance.

The authors argue that the global food trade involves many stakeholders (importers, exporters, freight forwarders, port authorities, regulators, and insurers) and that current paper-based certificate processes are slow, costly, and vulnerable to manipulation or error. By using blockchain as a tamper-proof distributed ledger, combined with InterPlanetary File System (IPFS) for storing larger files, they propose a system where food-related certificates (e.g., health certificates, bills of lading, inspection documents) can be digitally signed, stored, and verified in a transparent and auditable way (George & Al-Ansari, 2023).

Their literature review highlights several potential advantages of blockchain-based certification for food systems:

- Risk mitigation and authenticity: Immutable records and digital signatures help ensure that certificates and provenance data have not been altered, reducing fraud and mislabeling.
- Reduced transaction time and cost: Replacing manual, paper-based processes with smart contracts and shared ledgers can streamline document checks and approvals.
- Improved traceability and recall: End-to-end visibility from origin to port can support faster recall and investigation in case of safety issues.
- Tokenization and new financing models: Tokenization of assets (e.g., shipments, vessels) can create new opportunities for financing and risk-sharing in the supply chain.

Although GM-Ledger is designed for international trade certificates rather than household-level food assistance, it demonstrates that web3 / blockchain solutions are technically viable and can support secure, auditable flows of food-related information when social, legal, and policy frameworks are aligned (George & Al-Ansari, 2023). For a neighborhood-scale digital aid platform, similar ideas might be adapted in a lighter form, for example, to guarantee voucher authenticity, prevent double spending at partner supermarkets, or support auditable reporting to donors and regulators.

2.6 Existing Digital Aid Platforms and Their Limitations

Several established digital assistance systems inspired the Nubl concept, each contributing valuable practices but also revealing essential gaps. WFP SCOPE, the World Food Programme's e-voucher platform, demonstrates strong scalability and high security, enabling millions of beneficiaries to receive food assistance digitally. However, its architecture is optimized for large humanitarian organizations and national-level operations, making it less adaptable to small, community-driven environments. Similarly, the UNHCR Cash Assistance System provides dignity-preserving financial aid through secure distribution channels, yet it offers limited item-level transparency, making it difficult for donors or administrators to track how assistance is utilized at the grocery-basket level.

Other NGO solutions, such as RedRose ONEsolution, offer robust digital voucher management with strong data protection policies. While widely used by international NGOs, these systems focus on organizational control rather than community participation; they do not provide mechanisms for direct engagement between local donors, neighbourhood stores, and beneficiaries. Finally, commercial pay-it-forward applications enable individuals to anonymously sponsor grocery items for people in need. Despite their simplicity and popularity, these apps often lack essential operational features such as provider dashboards, real-time audit trails, eligibility verification, and structured approval workflows.

Compared with these platforms, Nubl addresses a different context and scale. It explicitly connects individual donors Gracious Neighbors, local beneficiaries, and participating neighborhood providers within a unified digital ecosystem. By incorporating privacy-by-design principles, NUBL minimizes personal data collection and uses pseudonymous identifiers to protect beneficiary dignity. Its real-time auditability records every request, approval, and redemption, enabling transparent reporting to donors and regulators. The system's fair-allocation model ensures equitable distribution of funds, while community-driven engagement features (for example non-intrusive donor badges) encourage giving without revealing sensitive information.

However, none of the existing digital aid platforms fully address the needs of a local, city-level food assistance ecosystem that integrates privacy-by-design, donor transparency, small-store participation, and real-time supermarket integration. This gap highlights the importance of NUBL's design as a localized, secure, and community-focused alternative.

2.7 Synthesis and Research Gap

The reviewed literature highlights three major supports that inform the design of modern digital assistance systems. First, studies on digital payments in humanitarian settings demonstrate that wallet-based transfers, e-vouchers, and digital cash systems are both feasible and effective even for low-literacy and highly vulnerable populations. Evidence from Afghanistan and Pakistan shows significant improvements in food security, convenience, and mental well-being, along with minimal diversion and lower operational costs. These findings confirm the potential of digital channels as a dignified and reliable method for delivering essential support.

Second, research on privacy-by-design and stakeholder engagement emphasizes that technical efficiency alone is insufficient for sustainable digital aid. Privacy risks, unclear roles, and inadequate user awareness can undermine trust and limit adoption. Frameworks such as those proposed by Carboni et al. highlight the need to embed privacy protections throughout the system lifecycle, define stakeholder roles clearly, and ensure that both beneficiaries and administrators understand how data is used and safeguarded.

Third, the emerging use of blockchain and distributed ledger technologies in food systems demonstrates the value of tamper-proof records, verifiable certificates, and transparent data-sharing. Although current applications such as GM-Ledger focus primarily on large-scale international trade, their principles offer insights into how small-scale social systems might achieve greater trust, auditability, and fraud prevention.

When examined together, these streams of literature reveal a clear pattern: the global ecosystem contains large-scale digital aid platforms (WFP SCOPE, UNHCR Cash Assistance, RedRose ONEsolution) and specialized technological frameworks (privacy-by-design models,

blockchain-based traceability). However, none of these systems fully address the needs of a localized, city-level digital food assistance model. In particular, the literature lacks solutions that:

- Combine digital payments, privacy-by-design, and trust/traceability mechanisms into a unified architecture.
- Support small supermarkets, local donors, and individual households rather than large NGOs.
- Provide real-time transparency, community engagement features, and item-level traceability.
- Fit Gulf-region socio-cultural and regulatory contexts, which differ significantly from the contexts studied in the existing literature.

This creates a clear research and system design gap. Current digital aid technologies are either too large-scale, too generic, or too limited in privacy and transparency to support the specific goals of a community-centered aid ecosystem.

Nuble aims to fill this gap by offering a privacy-preserving, locally deployable digital food assistance platform that integrates donors, beneficiaries, and neighbourhood providers with real-time auditability, pseudonymous identifiers, fair allocation rules, and transparent transaction flows features largely absent from existing literature and systems.

3. Methodology

3.1. Overview

In addition to the Systematic Literature Review (SLR) methodology used in Section 2 to ground the project in existing research (Kitchenham, 2007), the team needed a straightforward software development process to guide the practical implementation of the Nubl Platform. Because this is a two-semester capstone project with a fixed deadline, formal milestones, and strict documentation requirements (SRS, design, implementation, testing), we prioritized a methodology that:

- Supports clear, sequential deliverables for academic evaluation.
- Minimizes uncontrolled scope changes during the semester.
- It is easy to communicate with supervisors and external stakeholders.

After reviewing several standard software development lifecycle (SDLC) models, the team selected a Waterfall-based approach with limited feedback loops as the primary development methodology for the NUBL Platform.

3.2. Methodologies To Consider

Before choosing Waterfall, the team critically examined other common SDLC models and evaluated their suitability for this specific capstone context.

3.2.1 Agile / Scrum

Agile and Scrum emphasize short, iterative sprints, continuous delivery, and rapid response to changing requirements. This is attractive for industry projects with evolving product backlogs and close collaboration with customers.

However, for our project, Agile presented several challenges:

- Limited availability of stakeholders: We do not have a dedicated product owner or real end-users attending sprint reviews every 1–2 weeks. Most feedback comes during scheduled meetings with our supervisor.
- Fixed academic scope and deadlines: The capstone rubric specifies deliverables (SRS, design, prototype, final system) at particular weeks. Large mid-semester scope changes are not encouraged.
- High overhead for a small team: Proper Agile requires backlog grooming, sprint planning, reviews, and retrospectives. For a small student team with limited time, these ceremonies risk becoming overhead rather than added value.

For these reasons, pure Agile/Scrum was judged unsuitable as the primary methodology, although we still adopt some Agile ideas informally (For example, short internal check-ins and incremental commits).

3.2.2 Iterative / Incremental Model

The iterative/incremental model would allow us to build Nubl in repeated cycles: implement a subset of features, test them, then extend the system in later iterations.

This approach has advantages:

- Early partial prototypes can be shown to stakeholders.
- Risky components can be implemented earlier and improved over time.

However, we identified two main drawbacks in our context:

- Time constraints: Performing several complete build–test–refine cycles is complex within the limited duration of Capstone I and Capstone II.
- Documentation alignment: The academic evaluation expects relatively stable requirements and architecture from early in the project. Frequent significant changes across iterations could create inconsistencies in the SRS, design diagrams, and test plans.

Therefore, a fully iterative approach was considered too heavy for our constraints, although we still revisit and refine some design decisions when necessary.

3.2.3 Spiral / Risk-Driven Methods

Spiral and other risk-driven models are powerful for high-risk, safety-critical, or extensive systems. They emphasize continuous risk assessment, prototyping, and stakeholder evaluation in each spiral.

In our case:

- The Nubl Platform has essential privacy and ethical considerations, but it is not a safety-critical system, such as aviation or medical devices.
- The project size and team size are relatively small.
- The complexity of formally applying Spiral (full risk models, extensive prototyping cycles) is not justified for an undergraduate capstone with limited time and resources.

As a result, Spiral was acknowledged but not adopted as the main SDLC.

3.2.4 Waterfall Model

The Waterfall model is a linear and sequential development process in which each phase must be completed before moving to the next. Typical phases include requirements analysis, system design, implementation, testing, and deployment.

The Waterfall model offers several advantages that align well with the Nubl Platform's context:

- Clear phase boundaries and well-defined deliverables, which support academic evaluation and documentation requirements.
- Strong emphasis on upfront requirements and design, which is suitable for projects with a fixed scope.
- Ease of planning and progress tracking, particularly for small teams.
- Compatibility with formal documentation standards such as SRS, design specifications, and test plans.

However, the Waterfall model also has limitations:

- Limited flexibility to accommodate major requirement changes once development has progressed.
- Reduced ability to incorporate frequent stakeholder feedback compared to iterative approaches.

Despite these limitations, the Waterfall model was considered a strong candidate due to the project's fixed academic scope, structured deliverables, and limited stakeholder availability during the semester.

3.3. Our Methodology

Following this Waterfall-based approach, the Nubl Platform progresses through the following phases.

3.3.1 Requirements Analysis

Objectives

- Capture the goals and constraints of a city-level digital food assistance platform.
- Identify all stakeholders (Gracious Neighbors, Nubl Neighbors, Nubl Bridges, Admins, and municipal/charity partners).
- Define detailed functional requirements (FRs) and non-functional requirements (NFRs), including privacy-by-design, security, fairness, and usability.

Activities

- Conducted brainstorming sessions and informal interviews with potential users (e.g., students, community members, mentors).
- Analyzed relevant standards and regulations (GDPR, PDPL, ISO/IEC 25010) to derive privacy and quality attributes.
- Based on the literature (Section 2), aligned requirements with proven patterns in digital aid and privacy-by-design.
- Documented all requirements in a structured SRS using IEEE/ISO templates.

Deliverables

- Software Requirements Specification (SRS) including case models, user stories, and FR/NFR tables.
- Stakeholder and context diagrams for NUBL.

3.3.2 System and Architectural Design

Objectives

- Translate the SRS into a concrete system architecture that can be implemented with modern web technologies.
- Incorporate privacy-by-design and clear separation of roles (donors, beneficiaries, providers, admins).

Activities

- Designed high-level architecture diagrams (For example, client–server structure, integration with MyFatoorah, and optional blockchain/verifiable record components).
- Created ER diagrams for the database and defined main entities (users, households, requests, vouchers, transactions, supermarkets).
- Developed UML class diagrams, sequence diagrams, and UI wireframes to show the flow from request to redemption.
- Applied the quality attributes from ISO/IEC 25010 (e.g., security, reliability, usability) to influence design decisions.

Deliverables

- Architecture document and UML model set (use case, class, sequence, deployment).
- Database schema and initial UI mock-ups.

3.3.3 Implementation

Objectives

- Build the first working prototype of the Nubl web platform according to the design.

Activities

- Implemented the front-end using modern web frameworks and responsive design to support both desktop and mobile users.
- Implemented the back-end services and database layer, including role-based access control and logging.
- Integrated MyFatoorah for secure payment processing and implemented QR-based voucher redemption at participating providers.
- Followed coding standards, version control (For example, Git), and peer review within the team.

Deliverables

- Functioning prototype covering core flows: registration, request submission, approval, funding, and supermarket redemption.
- Source code repository with commit history and documentation.

3.3.4 Testing

Objectives

- Verify that the implemented system satisfies the agreed requirements and behaves reliably under typical usage conditions.

Activities

- Derived test cases directly from the FRs and use cases (requirements-based testing).
- Performed unit tests on critical components (For example, request validation, wallet balance checks, and access control).
- Conducted integration tests across the end-to-end flow: a Nubl Neighbor submits a request; a Gracious Neighbor funds it; a Nubl Bridge or supermarket redeems via QR; the donation is recorded in reports.
- Used test data to simulate different scenarios (multiple families, limited vouchers, insufficient balance, etc.).

Deliverables

- Test plan and test case specifications.
- Test execution report summarizing passed/failed cases and any fixes applied.

3.3.5 Evaluation and Reflection

Objectives

- Evaluate the system against project objectives (privacy, fairness, transparency, local impact).
- Reflect on limitations and future enhancements.

Activities

- Demonstrated the system to the supervisor and selected peers and collected qualitative feedback on usability and perceived trust.
- Assessed how well the implemented features align with the literature-based design principles from Section 2.
- Identified gaps (For example, limited real-user pilot, partial implementation of blockchain/verifiable records) to be addressed in future work.

Deliverables

- Evaluation section in the report, including a discussion of strengths, weaknesses, and lessons learned.

3.4. Project Plan

The Nubl Capstone I project follows the Waterfall methodology, where progress flows sequentially through structured phases. In this first phase, the focus is on planning, analysis, and documentation, while the system implementation and testing are scheduled for Capstone II. Each milestone in Capstone I corresponds to a primary deliverable defined in the Table of Contents.

Milestone	*Goal*	*Planned Work*	*Deliverables*
Milestone 1 Documentation & Planning	Establish the foundation of the project through research and documentation.	• Define the project purpose, problem statement, and objectives. • Develop the project scope and proposed solution summary. • Conduct the literature review to analyze existing systems and identify research gaps. • Select and document the development methodology (Waterfall).	Draft of Sections 1–3 (Introduction & Background, Literature Review, and Methodology).
Milestone 2 System Analysis	Translate the conceptual plan into detailed analytical and design artifacts.	• Identify system boundaries, leading actors, and stakeholders. • Specify functional and non-functional requirements. • Create UML diagrams such as use-case, activity, and sequence diagrams to represent system flows. • Validate requirements through internal team review.	Section 4 (System Analysis) — Requirements and Design Package.
Milestone 3 Preparation for Capstone II (Build Phase)	Finalize all documentation and prepare the system analysis package for the implementation phase.	• Consolidate all Capstone I deliverables and supervisor feedback. • Finalize decisions related to architecture, features, and data design. • Prepare the Capstone II project plan for development, testing, and deployment.	Final Capstone I Report, presentation slides, and complete system documentation to be used in Capstone II.

Table 5: Project Plan

3.4.1 Full Work Breakdown Structure

Due to the size of the complete Work Breakdown Structure, a simplified overview is presented in this section for readability.
The full interactive WBS is publicly available at: **WBS**

Figure 1: Full WBS Picture (3.4.1)

3.4.1.1 Work Breakdown Structure For Milestone 1

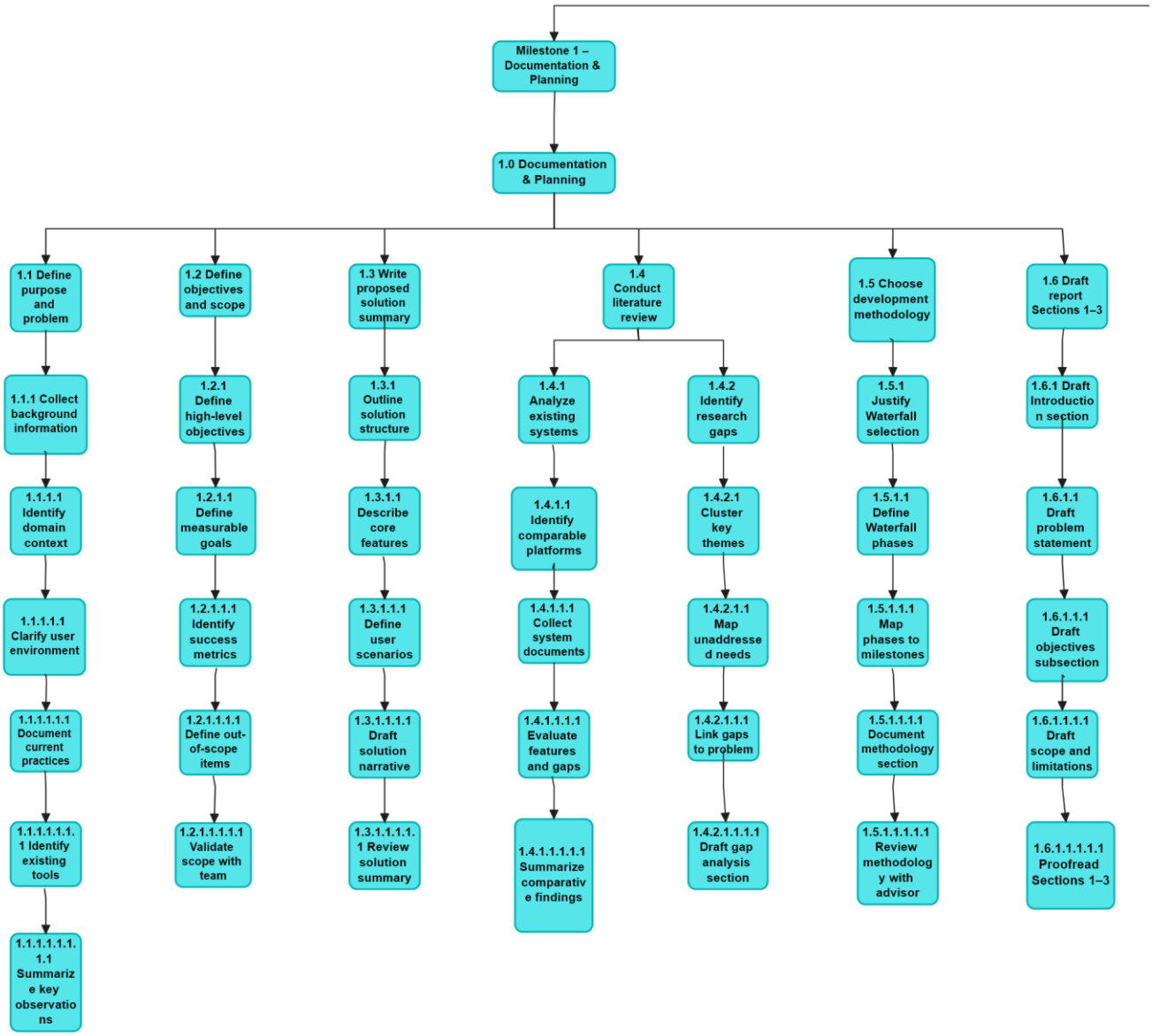


Figure 2: Work Breakdown Structure For Milestone 1 (3.4.1.1)

3.4.1.2 Work Breakdown Structure For Milestone 2

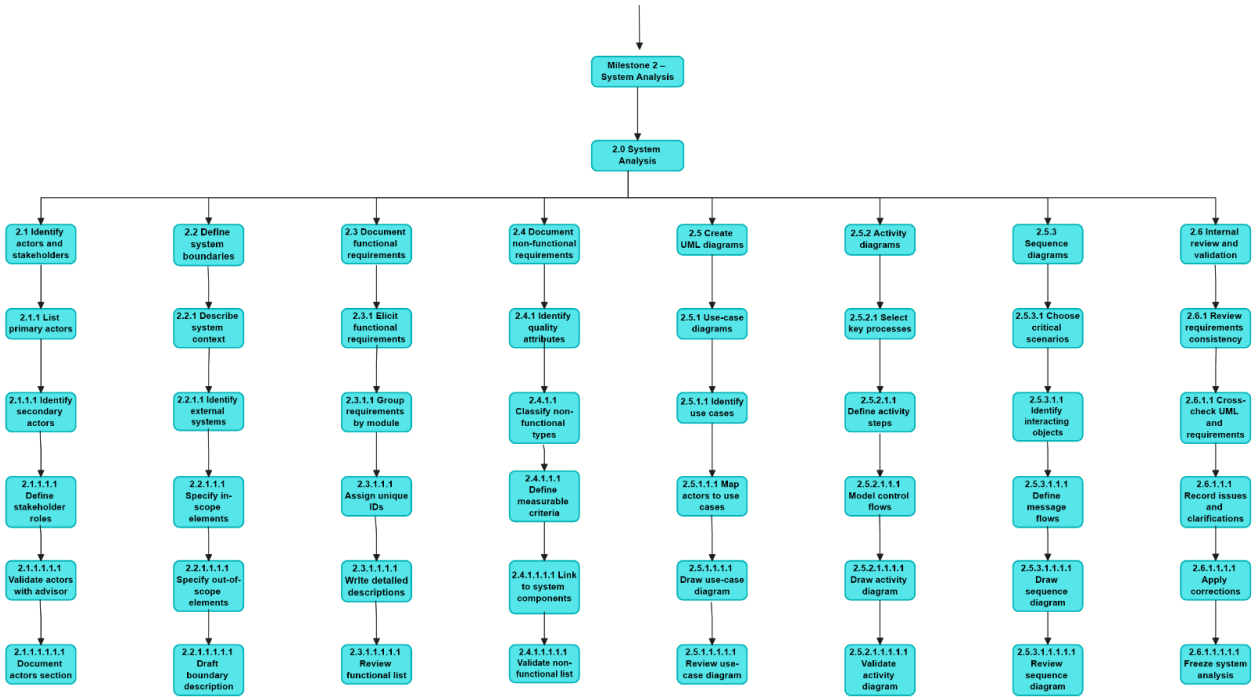


Figure 3: Work Break Down Structure For Milestone 2 (3.4.1.2)

3.4.1.3 Work Breakdown Structure For Milestone 3

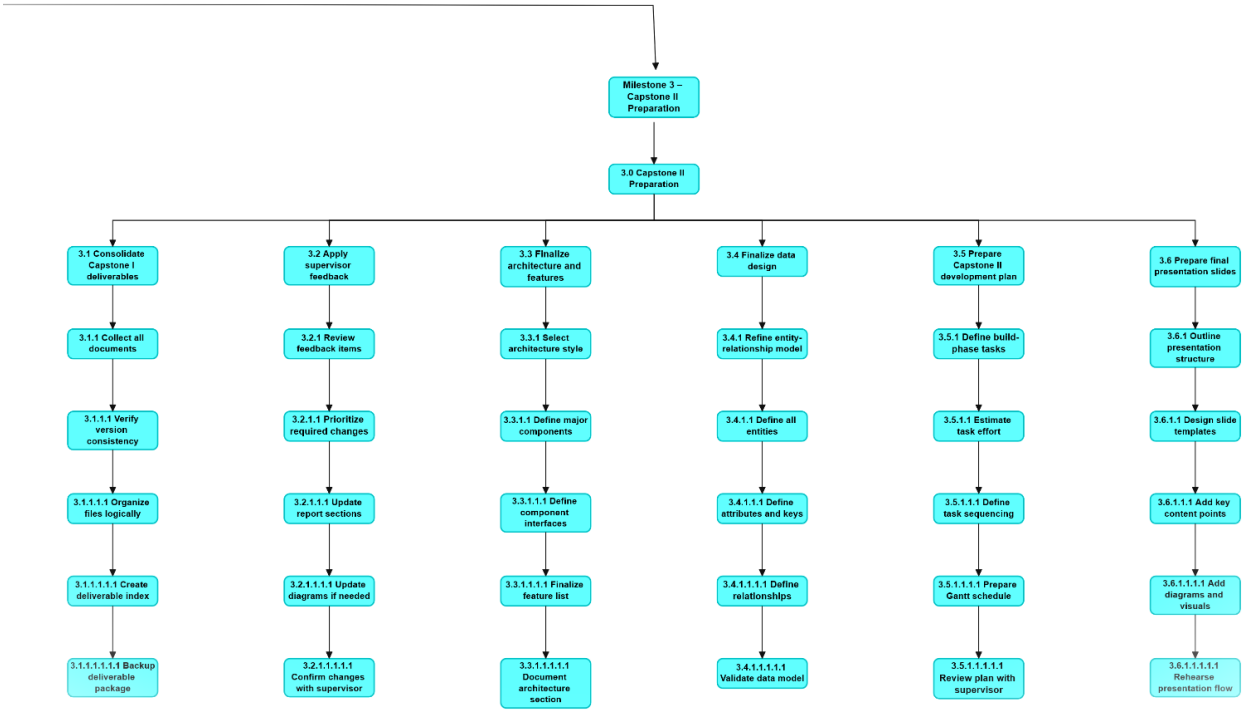


Figure 4: Work Break Down Structure For Milestone 3 (3.4.1.3)

4. System Analysis

This section specifies the functional and non-functional requirements for the Nubl Platform. The requirements have been elicited, analyzed, and documented in accordance with the international standard for requirements engineering, ISO/IEC/IEEE 29148:2018. This ensures that each requirement is unambiguous, testable, and traceable.

The quality characteristics targeted by these requirements are based on the ISO/IEC 25010:2023 Systems and Software Quality Model. Key areas of focus include:

- Functional Suitability: Ensuring the system provides a complete and appropriate set of features.
- Performance Efficiency: Defining strict targets for response time and resource utilization under load.
- Reliability: Guaranteeing high availability and robust data integrity.
- Usability: Ensuring the platform is intuitive and efficient for end-users.
- Security: Implementing comprehensive measures to protect data in transit and at rest.

All privacy-related requirements adhere to the principles and regulations set forth by the Saudi National Cybersecurity Authority (NCA) Data Privacy Law (PDPL).

4.1 Overview

Nubl Platform is a digital humanitarian system that connects donors, beneficiaries, and service providers within a hyper-local, neighborhood-based ecosystem. The goal is to make essential food assistance easy, fast, and dignified.

This section analyzes the Minimum Viable Product (MVP), a city-based web platform that operates on a neighborhood level, focusing on grocery-first aid with strict privacy-by-design principles.

The system includes the following modules:

1. Authentication & KYC-Light Verification
2. City Management Module
3. Beneficiary Request Module (for groceries)
4. Fund Allocation & Admin Exception-Handling System
5. Provider Fulfillment (QR-Based)
6. Gracious Neighbor (Sponsor) Dashboard – aggregated data only
7. Admin Console – system and data management
8. Audit & Reporting Module with secure Role-Based Access Control (RBAC)

The MVP is limited to one city, subdivided into registered neighborhoods, and integrates only one payment gateway (MyFatoorah). The focus is strictly on essential items from supermarkets

and grocery stores, excluding restaurants in this phase. Personal data storage is minimized. System constraints include limited resources.

4.2 System Requirement Sources

The system requirements come from reliable and ethical sources to make sure they are realistic, legal, and useful:

Source Type	***Description***
Stakeholder Meetings	Interviews with the supervisor (Dr. Basit Shahzad), Al-Bir Society in Madinah, team members, and users who represent donors, recipients, providers, and biralmadinah, who will be our sponsor.
Document Study	Analysis of existing global systems such as Barakah, Ehsan, WFP SCOPE, National Platform for Fundraising, RedRose, and UNHCR Cash Assistance to learn best practices.
Technical Resources	Bonat digital voucher and QR-based redemption workflows, MyFatoorah API documentation, secure web standards (HTTPS/TLS), and QR-code libraries.
Ethical & Legal Standards	Saudi NCA Data Privacy Law and GDPR guidelines for protecting personal information.
Research & Observation	Study of humanitarian aid systems that value dignity and privacy (Richey et al., 2024), and Cash and Voucher Assistance CVA.
Security and privacy in implementation	References from OWASP Top Ten 2025 and Application Security Verification Standard ASVS

Table 6 : System Requirement Sources

All these sources helped the team define requirements that are practical, fair, and privacy-focused

4.3 Functional Requirements (FRs)

These functional requirements form the core behavior of the MVP and will serve as the base for Capstone II design and testing. And The functional requirements are documented using IEEE-compliant system-level statements and are explicitly linked to their primary actors to reflect the user interactions derived from the use case model.

ID	*Actor*	*Requirement Statement*	*Rationale*	*Verification Method*	*Source*
FR-1.1	All Users	The system shall allow users (Donors, Providers, Beneficiaries,	Provides the entry point for all user roles to access the system.	Functional test of registration flow.	Stakeholder Meeting

		Admins) to register.			
FR-1.2	All Users	The system shall enforce One-Time Password (OTP) verification during registration.	Enhances security by verifying the user's phone number.	OTP delivery and validation test.	Tech Ref (FR-AUTH-01)
FR-1.3	All Users	The system shall validate all user phone numbers against the Saudi format ^966[0-9].	Ensures data consistency and successful OTP delivery in the target region.	Input validation test for phone field.	Stakeholder Meeting
FR-1.4	Admin	The system shall require Beneficiary and Provider accounts to be approved by an Admin before granting full access.	Ensures only vetted Beneficiaries and Providers can participate in transactions.	Test access control for pending accounts.	Stakeholder Meeting
FR-1.5	System (Internal)	The system shall enforce Role-Based Access Control (RBAC) to restrict user access to authorized data only.	Protects data integrity and privacy by isolating roles.	RBAC access-control tests for each role.	Tech Ref (FR-AUTH-04/05)
FR-2.1	Beneficiary, Provider	The system shall perform a KYC-Light identity	Verifies user legitimacy while	Review of data collection fields.	Legal Policy + FR-AUTH-03

		verification using only non-sensitive data.	minimizing PII collection per privacy-by-design.		
FR-2.2	System (Internal)	The system shall use Row-Level Security (RLS) in the database to protect user records.	Provides an additional layer of data protection at the database level.	Database policy inspection and compliance test.	FR-PV-03
FR-3.1	Donor	The system shall process donations through the MyFatoorah API.	Enables secure and standardized online payment processing.	Functional payment test.	Technical Ref
FR-3.2	Donor	The system shall generate a receipt for each successful payment.	Provides donors with proof of their contribution for their records.	Receipt generation and content verification.	Stakeholder Req.
FR-3.3	System (Internal)	The system shall log all payment transactions in the audit trail.	Ensures all financial activities are traceable and auditable.	Audit log review after a test transaction.	Technical Ref
FR-3.4	Donor	The system shall display a retry message if the payment API is unavailable or returns an ambiguous response.	Informs the user of a temporary issue without causing confusion.	Failure-mode test simulating API outage.	Technical Ref

FR-3.6	System (Internal)	The system shall log a critical failure if the payment does not succeed.	Alerts administrators to persistent issues with the payment gateway.	Log inspection after a permanent failure test.	Technical Ref
FR-4.1	Donor	The system shall display aggregated statistics on the Donor dashboard (e.g., funds raised, items delivered).	Provides transparency to donors on the impact of their contributions.	Dashboard data accuracy verification.	Privacy Standard + FR-SP-01
FR-4.2	System (Internal)	The system shall ensure that no Personally Identifiable Information (PII) is revealed in aggregated dashboards or exports.	Protects the privacy of beneficiaries and providers.	Attempt to access PII returns 403; CSV audit.	FR-PV-01
FR-5.1	Beneficiary	The system shall allow Beneficiaries to submit digital requests for items from a Provider's menu.	Enables a structured and controlled digital request workflow.	Form submission and DB record creation check.	User Story + FR-BE-04
FR-5.2	Beneficiary	The system shall validate request form inputs (e.g., item type, quantity)	Prevents invalid data from entering the system.	Form validation and boundary tests.	User Story

		before submission.			
FR-6.1	System (Internal)	The system shall automatically check a beneficiary's available allowance limit upon request submission.	Prevents users from exceeding their designated support limits.	Functional test of allowance limit check.	Business Rule + FR-AL-01
FR-6.2	System (Internal)	The system shall automatically check for available funds in the relevant fund pool upon request submission.	Ensures requests are only processed if funds are available to cover them.	Functional test of fund availability check.	Business Rule + FR-AL-02
FR-6.3	Beneficiary	The system shall display the message “Your request exceeds your available allowance for this period” if the limit is exceeded.	Provides clear, user-friendly feedback on validation failure.	UI test for over-limit scenario.	Business Rule
FR-6.4	System (Internal)	The system shall queue a request for retry within 60 seconds if it fails due to temporary fund unavailability.	Improves user experience by automaticall y handling temporary funding gaps.	Retry queue simulation test.	Business Rule

FR-7.1	Admin	The system shall provide an interface for Admins to view, approve, or reject pending or flagged requests.	Gives administrators manual oversight and control over the workflow.	Admin UI functional test.	Supervisor Review + FR-AD-03
FR-7.2	Admin	The system shall record the Admin's decision, ID, and timestamp in the audit log.	Ensures all manual interventions are accountable and traceable.	Audit record verification after manual action.	FR-AD-04
FR-8.1	System (Internal)	The system shall generate a unique, one-time-use QR token for each approved request.	Provides a secure and simple mechanism for redeeming requests.	QR code generation unit test.	Technical Ref + FR-AL-03
FR-8.2	System (Internal)	The system shall set a default Time-To-Live (TTL) of 3 hours for each QR token.	Prevents old, unused requests from remaining active indefinitely.	TTL expiry simulation test.	Technical Ref + FR-AL-04
FR-8.3	Admin	The system shall allow the QR token TTL to be configurable by an Administrator.	Provides flexibility to adjust redemption windows based on operational needs.	Test TTL change via admin setting.	Technical Ref
FR-9.1	Provider	The system shall allow Providers to	Offers flexibility for	Functional QR	Provider Workshop + FR-PR-02

		redeem QR codes via scanning or manual entry.	providers based on their available hardware.	redemption test.	
FR-9.2	Provider	The system shall return an error for invalid or expired codes within ≤ 1 second.	Provides immediate feedback to the provider on failed redemptions.	Invalid token and latency simulation.	Provider Workshop
FR-9.3	Provider	The system shall allow two retries within 30 seconds on a failed redemption attempt.	Handles transient network or scanning errors gracefully.	Retry simulation test.	Provider Workshop
FR-10.1	Provider	Providers shall upload proof of fulfillment (receipt/photo) after each request.	Provides evidence of delivery for auditing and accountability.	File upload functional test.	Admin/Audit Req + FR-PR-03
FR-10.2	System (Internal)	The system shall enforce a maximum file size of 5 MB for proof of fulfillment uploads.	Prevents excessive storage consumption.	File size boundary test.	Admin/Audit Req
FR-11.1	System (Internal)	The system shall update fund and allowance balances atomically after a	Ensures financial consistency by preventing partial updates.	Transaction rollback simulation.	Business Rule + FR-PR-04

		successful redemption.			
FR-11.2	System (Internal), Admin	The system shall notify Admins via the dashboard and email within 60 seconds of the transaction outcome.	Alerts administrators to critical financial transactions.	Email and dashboard alert verification.	Business Rule
FR-12.1	Admin	The system shall provide Admins with full CRUD management for users, funds, and policies.	Facilitates efficient system administration.	Functional UI tests for each CRUD operation.	Supervisor + FR-AD-01/02/03
FR-12.2	System (Internal)	The system shall reflect policy changes for all users within ≤ 5 seconds.	Ensures policy enforcement is timely and consistent.	Performance measurement of policy propagation.	FR-AD-05
FR-13.1	System (Internal)	The system shall record every transaction state change with the actor ID and timestamp.	Provides a clear, chronological history of every request.	Audit export validation.	Security Req + FR-AU-01/02
FR-13.2	System (Internal)	The system shall include a SHA-256 hash with each audit log entry.	Provides tamper-evident traceability for compliance and security.	Hash integrity check on exported logs.	FR-AU-03

FR-14.1	System (Internal)	The system shall send critical emails within ≤ 30 seconds.	Ensures reliable and timely communication of important events.	Email delivery time measurement.	Usability + Stakeholder Input
FR-14.2	System (Internal)	The system shall retry sending a failed email up to 3 times before alerting an Admin.	Improves reliability of notifications against transient SMTP failures.	SMTP failure test and retry log inspection.	Usability
FR-15.1	Admin	The system shall allow Admins to export audit and financial reports in CSV and PDF formats.	Supports offline auditing and financial accountability.	File export functional test.	Admin + FR-SP-03
FR-15.2	System (Internal)	The system shall generate reports with checksums within ≤ 5 seconds.	Ensures data integrity and provides responsive performance.	Export timing and checksum verification.	FR-AU-03
FR-16.1	System (Internal)	The system shall return an HTTP 403 Forbidden error for any unauthorized action.	Enforces security by denying access where not permitted.	Security penetration testing.	Security Req + FR-AUTH-04
FR-17.1	Admin	The system shall allow Admins to configure system-wide	Enables dynamic control over beneficiary allowances	CRUD and latency testing ≤ 5 s.	Business Rule + FR-PM-01

		allowance values.	based on policy.		
FR-18.1	All Users	The system shall support both English and Arabic user interfaces.	Improves accessibility for bilingual users in the target region.	UI language validation.	User Experience
FR-18.2	All Users	The system shall render Arabic content with a proper Right-to-Left (RTL) layout.	Ensures correct and natural display for Arabic-speaking users.	UI layout validation in Arabic.	User Experience
FR-19.1	System (Internal)	The system shall automatically generate weekly and monthly summary reports.	Provides periodic performance visibility to stakeholders without manual effort.	Scheduled job execution test.	Reporting + FR-SP-03
FR-20.1	Admin	The system shall allow Admins to deactivate or reactivate user accounts.	Supports security and full account lifecycle management .	Functional test of account status change.	Admin Control + FR-AD-01
FR-20.2	System (Internal)	The system shall immediately block login attempts from deactivated accounts.	Ensures that deactivation is an effective security control.	Login attempt test after deactivation.	Admin Control
FR-21.1	Provider	The system shall allow Providers to "adopt" a pending	Encourages voluntary provider participation	Functional adoption test.	Business Rule + FR-BE-08

		request and fulfill it as a donation.	and goodwill.		
FR-22.2	System (Internal)	The system shall credit the adopting Provider as the donor in the audit log.	Provides transparency and recognition for provider contributions.	Audit record check after adoption.	Business Rule
FR-23.1	Provider	Providers shall be able to toggle their service capacity (ON/OFF).	Balances provider workload and manages system availability.	UI toggle test.	Extended FR + FR-PM-02/03
FR-23.2	Provider	Providers shall be able to set custom pickup notes and operating hours.	Communicates important logistical information to beneficiaries.	Functional test of saving and displaying notes.	Extended FR
FR-23.3	System (Internal)	The system shall reflect capacity changes in the Beneficiary view within ≤ 1 second.	Ensures beneficiaries have real-time information on provider availability.	Timing measurement ≤ 1 s.	Extended FR
FR-24.1	Admin	Admins shall be able to pause the allocation engine globally or on a per-Provider basis.	Allows for controlled system maintenance and fund reconciliation.	Pause/resume integration test.	Extended FR + FR-AL-05

FR-24.2	System (Internal)	The system shall ensure no data loss occurs when the allocation engine is resumed.	Guarantees system reliability and data integrity during maintenance.	Queue state validation after resume.	FR-AD-03
FR-25.1	System (Internal)	The system shall minimize the storage of Personally Identifiable Information (PII).	Adheres to Privacy-by-Design principles and reduces risk.	Data schema review and penetration test.	Ethics / Legal + FR-PV-01
FR-25.2	System (Internal)	The system shall use pseudonymous IDs in place of real user IDs where possible.	Enhances user privacy by decoupling identity from activity.	Database inspection for pseudonymous keys.	FR-PV-02
FR-25.3	System (Internal)	The system shall encrypt all sensitive data at rest and in transit.	Protects user data from unauthorized access.	Security audit and configuration check.	Ethics / Legal

Table 7 : Functional Requirements (FRs)

4.4 Non-Functional Requirements (NFRs)

These define how the system should perform. They are measurable and can be tested or verified after development. Each non-functional requirement is explicitly classified according to the ISO/IEC 25010 quality characteristics to improve clarity, traceability, and evaluation.

ID	***NFR Type***	***Non-Functional Requirement***	***Target / Verification Method***	***Source***
NFR-1	Performance Efficiency	The system shall respond to any user request within ≤ 2	JMeter load testing	FR-1.1, FR-5.2, FR-6.1, FR-9.1, ISO/IEC 25010

		s (p95) at ≤ 100 concurrent users.		
NFR-2	Performance Efficiency	The system shall handle at least 100 orders per hour without degradation.	Load test; error rate ≤ 1%	FR-5.2, FR-8.1, FR-9.1, FR-11.1
NFR-3	Reliability / Availability	The system shall maintain 99.9% uptime, allowing ≤ 43.2 minutes of downtime per month.	Hosting monitor report	Infrastructure objective / System Ops
NFR-4	Security	The system shall ensure data integrity and encryption for all transactions and require a valid token for every API request.	Security validation; API & DB audit logs	FR-1.5, FR-16.1, FR-25.3
NFR-5	Scalability	The system shall support elastic auto-scaling and handle ≥ 400 active users.	Load test to 400 users; CPU ≤ 70%, Memory ≤ 75%	Architecture decision
NFR-6	Usability	The system shall allow a first-time user to complete registration or submit a request within 1–2 minutes without training.	Usability test; avg ≤ 2 min, success ≥ 95%	FR-1.1–1.5, FR-5.1–5.3, ISO/IEC 25010
NFR-7	Reliability / Integrity	The system shall persist data without loss or corruption under heavy load.	Stress test; DB consistency = 100%	FR-3.3, FR-5.2, FR-7.2, FR-11.1
NFR-8	Integrity / Auditability	All transactions shall be logged automatically and shall not be manually editable.	SHA-256 checksum verification	FR-13.2, FR-15.1
NFR-9	Reliability / Recoverability	The system shall perform automatic backups every 24 hours.	Backup timestamp verification	Operations policy
NFR-10	Security	All data communication shall use TLS 1.3 and data at rest shall use AES-256 encryption.	Security penetration testing	FR-25.3
NFR-11	Maintainability	The system shall be modular and containerized to allow independent deployment and scaling.	Architecture inspection during development	Architecture decision

Table 8 : Non-Functional Requirements (NFRs)

These NFRs ensure the system is reliable, secure, scalable, maintainable,

and consistently perform as expected.

4.5 Use Case Diagram

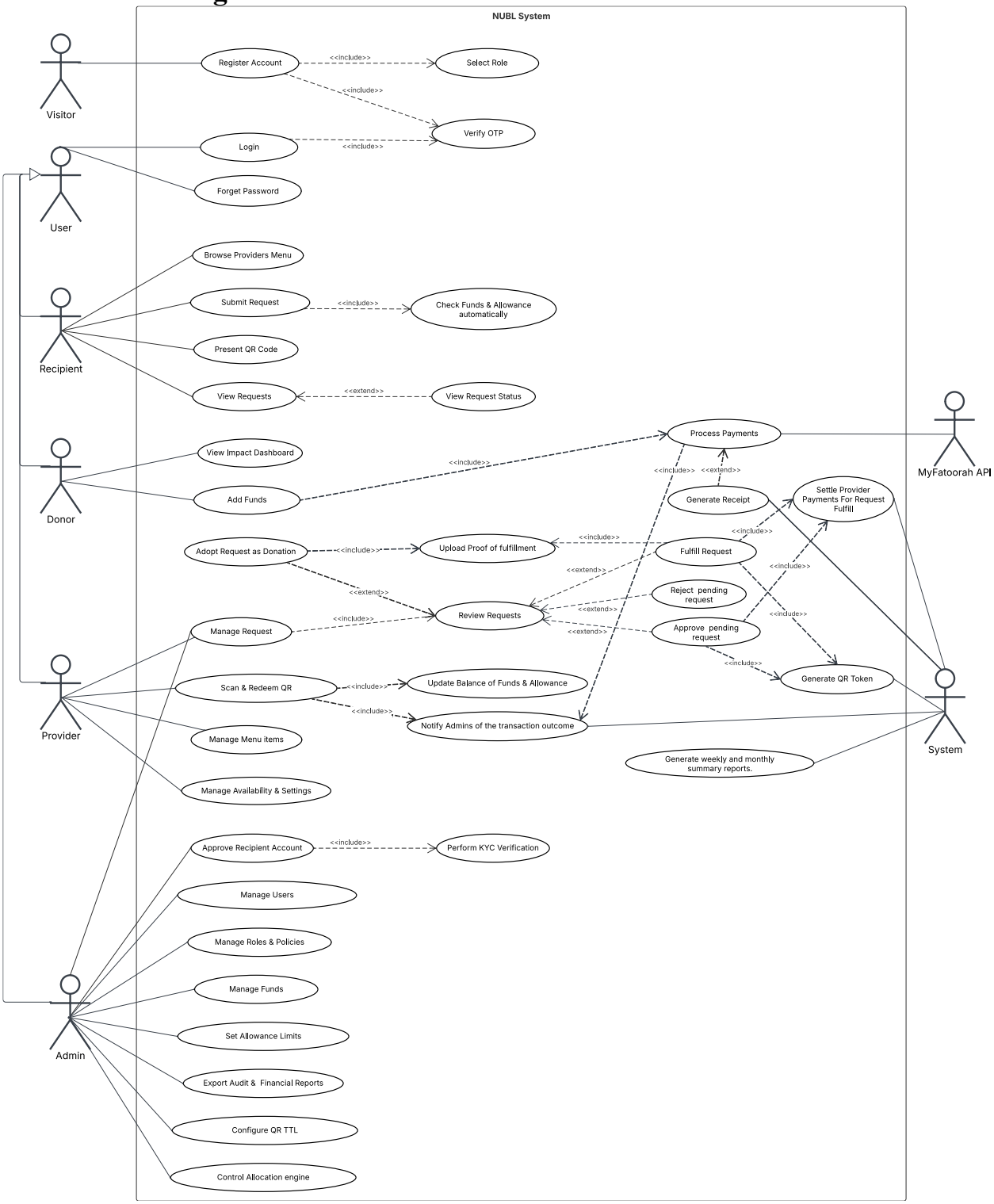


Figure 5: Use Case Diagram (4.5)

4.6 System Activity Flow Diagram

Figure 6: System Activity Flow Diagram (4.6)

5. System Design

5.1 Overview

We leverage a modern, scalable cloud architecture to ensure privacy, transparency, and efficiency in delivering neighborhood-based food assistance. The system is designed to securely connect donors (Gracious Neighbors), recipients (Nubl Neighbors), and providers (Nubl Bridges) through a robust digital workflow.

5.2 Candidate Architectures

5.2.1 Architecture A (Simple AWS Cloud Architecture)

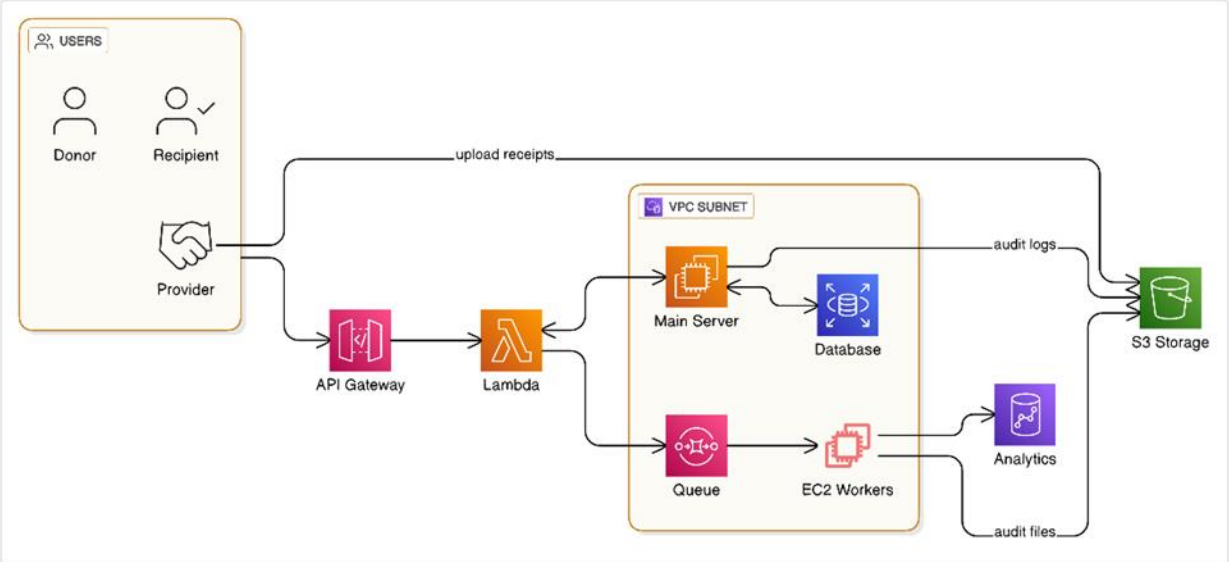


Figure 7: Architecture A (Simple AWS Cloud Architecture) (5.2.1)

5.2.1.1 Key Components

- API Gateway: Serves as the secure entry point for all client requests, routing traffic to backend services.
- AWS Lambda: Handles serverless business logic, such as processing requests, approvals, and QR code generation, ensuring scalability and cost efficiency.
- VPC Subnet: Hosts the core application infrastructure within a secure, isolated network environment.
    - Main Server (EC2): Runs the main application logic and interfaces with the database.
    - Database (RDS): Stores user data, transaction records, and audit logs with strong privacy controls.
    - Queue (Auto Scaling): Manages asynchronous processing of requests, approvals, and notifications, supporting high availability.
    - Compute Nodes (EC2 Workers): Dedicated worker instances (highlighted in red) process queued tasks, such as report generation and analytics
- S3 Storage: Securely stores uploaded receipts and audit files, ensuring traceability and compliance.
- Analytics (Redshift): Aggregates and analyzes system data to provide insights and trends for administrators and donors, without exposing personal information.

5.2.1.2 Workflow Summary

1. Users interact with the platform via the API Gateway, which forwards requests to Lambda functions.
2. Lambda processes business logic and communicates with the Main Server and Database.
3. Approved requests are queued for fulfillment; worker nodes process these tasks and update analytics.
4. Providers upload receipts to S3 after fulfilling requests; all actions are logged for transparency.
5. Aggregated analytics are generated for reporting and donor dashboards.

This architecture ensures that the Nubl Platform is secure, scalable, and maintains strict privacy-by-design principles, supporting the project's mission to deliver dignified, efficient, and transparent assistance at the neighborhood level.

5.2.2 Architecture B (AWS Cloud Architecture)

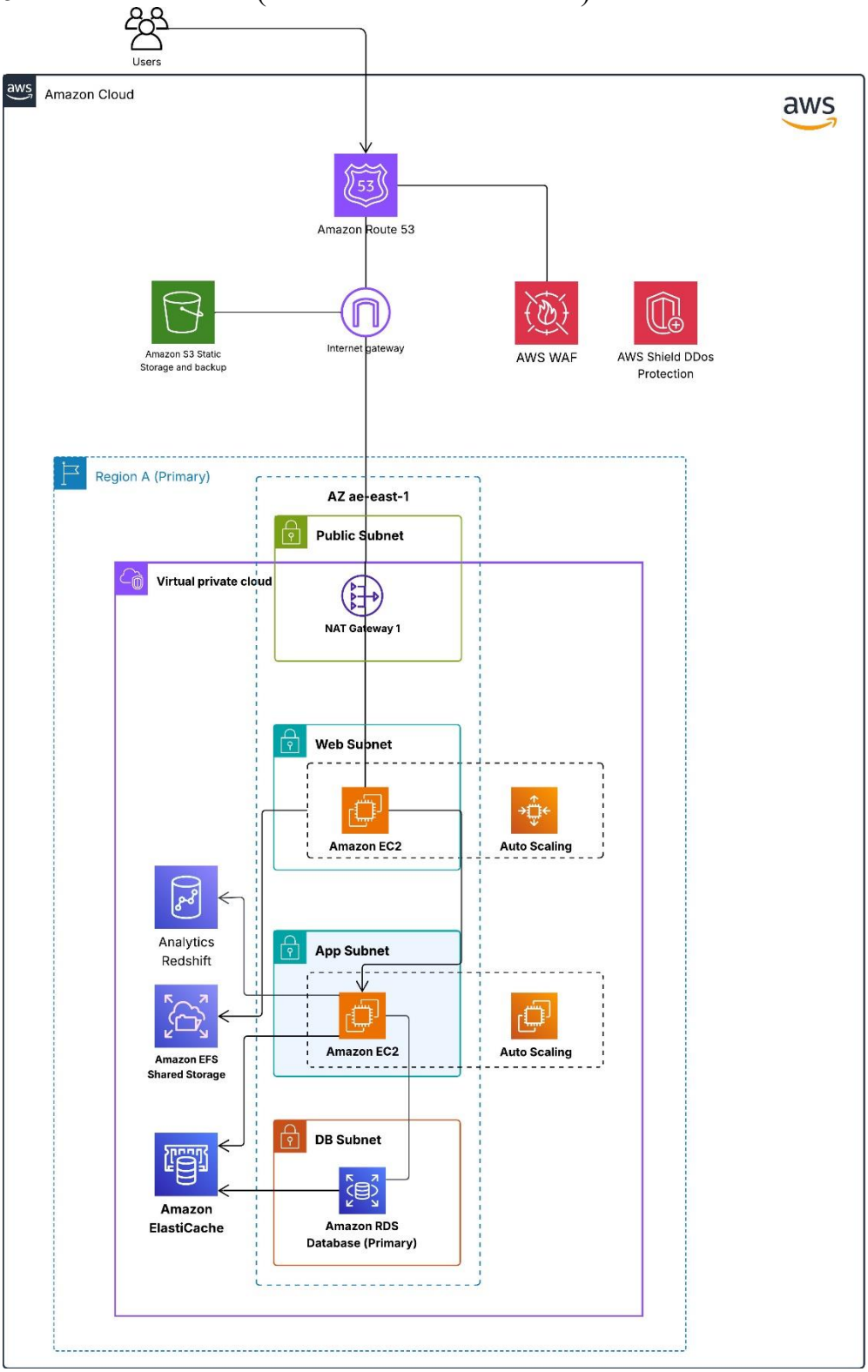


Figure 8: Architecture B (AWS Cloud Architecture) (5.2.2s)

5.2.2.1 Architecture Components

Security and Routing Layer

- Amazon Route 53: Provides DNS routing to direct users to the appropriate service endpoints.
- AWS WAF (Web Application Firewall): Protects the application from SQL injection, XSS, and other common web attacks.
- AWS Shield: Provides DDoS protection to ensure service availability.
- Amazon CloudFront (CDN): Caches and delivers static content (HTML, CSS, JS, images) from global edge locations.

5.2.2.2 Network and Infrastructure Layer

- Amazon VPC: Provides an isolated virtual network environment.
- Public Subnet: Hosts internet-facing components such as the NAT Gateway and the Load Balancer.
- Private Subnets (Web, App, DB): Hosts internal application servers and databases, protected from direct internet access.
- Multi-AZ Deployment: Ensures high availability and automatic failover across multiple Availability Zones.

5.2.2.3 Compute and Scaling Layer

- Elastic Load Balancer (ELB): Distributes incoming traffic across multiple EC2 instances.
- Auto Scaling Group (ASG): Automatically scales the number of EC2 instances based on load.
- Amazon EC2 Instances: Execute both the web tier and backend business logic.

5.2.2.4 Storage and Data Layer

- Amazon EFS: Provides shared file storage accessible by all EC2 instances.
- Amazon ElastiCache (Redis): Provides in-memory caching to reduce database load and improve performance.
- Amazon RDS (Primary/Secondary): Managed relational database with automated backups and replication.
- Amazon S3: Stores static assets, backups, and media files with high durability.

5.2.2.5 Request Processing Flow

1. The client request is routed through Route 53, then inspected by AWS WAF and AWS Shield.
2. The request reaches CloudFront, which serves cached static content when available.
3. CloudFront forwards dynamic requests to the Elastic Load Balancer (ELB).
4. ELB distributes traffic across EC2 instances in the Auto Scaling Group.
5. Application servers use ElastiCache for fast data access and EFS for shared storage.
6. Persistent data operations are handled by Amazon RDS.
7. The processed response returns to the user via the same path (EC2 → ELB → CloudFront → Route 53).

5.2.3 Architecture C (Architecture 3-Tier Web Design)

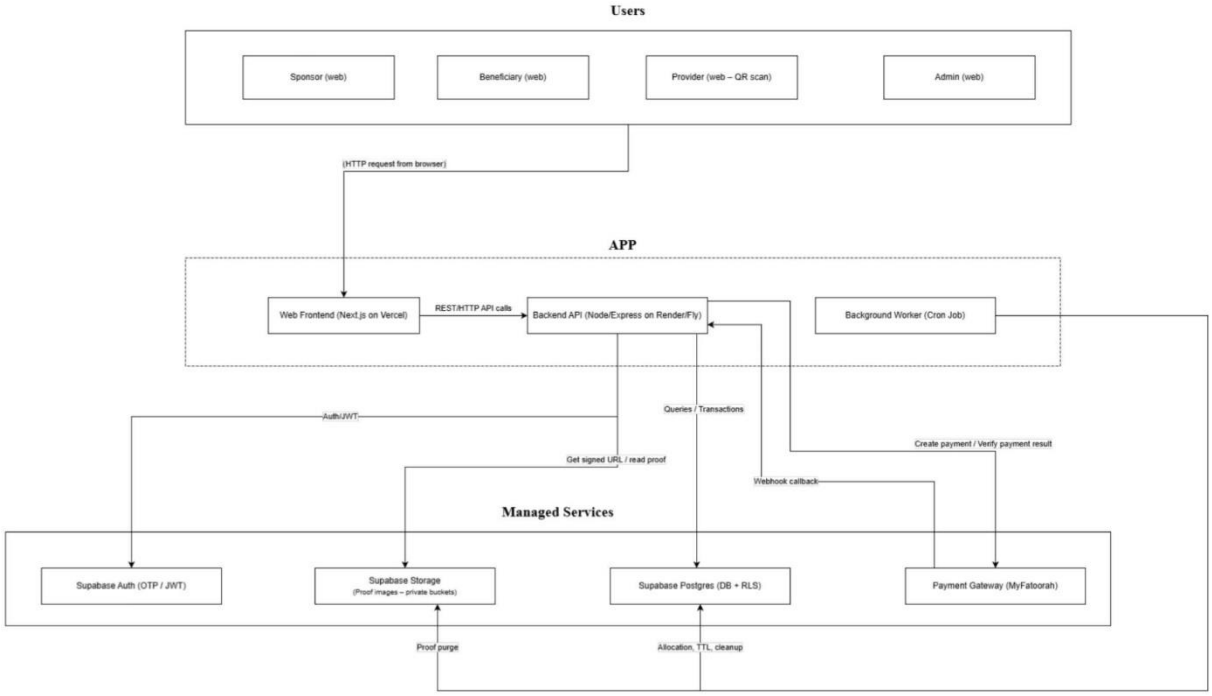


Figure 9: Architecture C (Architecture 3-Tier Web Design) (5.2.3)

The Nubl system adopts a managed three-tier web architecture composed of:
(1) Presentation Tier, (2) Application Tier, and (3) Data Tier.
This structure supports modularity, maintainability, security, and scalability while remaining appropriate for the project's MVP scope.

5.2.3.1 Presentation Tier (Frontend)

The Presentation Tier consists of a Next.js web application that provides role-based user interfaces for Sponsors, Beneficiaries, Providers, and Administrators.
All interactions with the system occur through HTTPS REST API calls.
User authentication and authorization are performed using JWT tokens issued by Supabase Auth.

5.2.3.2 Application Tier (Backend API and Worker)

The Application Tier is implemented using a Node.js/Express backend, hosted on a managed cloud platform.
This tier is responsible for:

- Enforcing authentication and role-based access control.
- Managing request creation, assignment, and fulfillment workflows.
- Generating and validating one-time QR tokens.
- Managing fund balances and allowance rules.
- Writing immutable audit logs.
- Integrating with MyFatoorah for payment processing.

A lightweight background worker executes scheduled tasks, such as token expiration, report generation, and removal of expired proof documents.
The backend is designed as a stateless API, enabling horizontal scaling if required.

5.2.3.3 Data Tier (Database, Storage, and Authentication)

The Data Tier uses managed cloud services to ensure data consistency, privacy, and availability:

- Supabase Postgres as the main relational database for system entities (users, requests, assignments, transactions, audit logs, etc.).
- Supabase Storage for secure handling of uploaded proof/receipt files.
- Supabase Auth for OTP-based authentication and JWT issuance.
- MyFatoorah as the external payment gateway used exclusively by the backend.

5.2.3.4 Architectural Summary

This architecture provides:

- Clear separation of concerns.
- Secure handling of sensitive data through RLS, JWT, and private storage buckets.
- Minimal infrastructure overhead by using managed cloud services.
- Practical feasibility for implementation by the project team within the Capstone timeline.
- Flexibility for future enhancements, including mobile applications or additional backend instances.

5.2.4 Evaluation of Candidate Architectures

This section presents an objective evaluation of the three proposed architectural designs. Each architecture was assessed based on feasibility, alignment with functional and non-functional requirements, implementation complexity, and suitability for a Capstone-level MVP.

A. Architecture A - Simple AWS Cloud Architecture

Strengths

- Suitable for an MVP implementation.
- Uses managed cloud services, minimizing DevOps overhead.
- Clear separation of concerns (API, worker, database, storage).
- Supports essential system features (auditing, logging, QR generation, fund handling, receipt uploads).
- Feasible for development by a three-member Capstone team.
- Extensible to future mobile applications.
- Fully aligned with core functional requirements.

Weaknesses

- Security components (For example WAF, CDN) are not included.
- High-availability configurations are not addressed (acceptable for MVP).

Overall Assessment

A balanced and realistic architecture appropriate for the scope of the system.

B. Architecture B - AWS Cloud Architecture

Strengths

- Professionally presented using enterprise-grade AWS services.
- Includes advanced capabilities such as WAF, Shield, CloudFront, RDS, and autoscaling.
- Suitable for large-scale production environments.
- Demonstrates solid understanding of cloud infrastructure.

Weaknesses

- Over-engineered for an MVP and Capstone scope.
- Not feasible to implement within one academic semester.
- More complex than necessary for the system's scale and auditability requirements.
- Introduces unnecessary complexity to payment and QR workflows.

C. Architecture C - 3-Tier Architecture

Strengths

- Clean and well-structured three-tier design.
- Uses Supabase and managed backend services aligned closely with system requirements.
- Can be quickly implemented for very small or simple projects.

Weaknesses

- NUBL involves complex workflows and sensitive operations, while Supabase is generally more suitable for lightweight or straightforward applications.
- The architecture relies heavily on Supabase-managed services, which reduces control over business logic, security enforcement, and system behavior, results in weak service layering, limits testability, increases maintenance complexity, and ultimately lowers the educational value expected from a capstone project.
- Scalability, availability, and reliability are clearly defined as NFRs in the SRS, yet this architecture does not address them in a clear or concrete way.
- Does not explicitly include routing/CDN components.
- Combining Supabase with an additional backend layer results in backend duplication, architectural ambiguity, and increased maintenance complexity.

Overall Assessment

Partially aligned with the functional requirements but shows clear limitations in meeting the defined NFRs. Its reliance on Supabase makes it less suitable for a complex system like NUBL, particularly in terms of backend control, scalability, and long-term maintainability.

Overall Assessment

Technically strong but impractical. Not aligned with the project's timeframe, team capacity, or MVP requirements.

D. Final Ranking

Rank	***Architecture***	***Reason***	***Evaluation Metrics***
1	Architecture A — Simple Managed AWS Cloud Architecture	Effective and fully suitable for MVP.	simple, fast, maintainable, scalable, available, performant, secure, cost-effective
2	Architecture B — AWS Cloud Architecture	Somewhat challenging for Capstone implementation due to cost, but it's feassssible.	Security, Availability, Scalability, Maintainability, Complexity, latency

3	Architecture C — Managed 3-Tier Architecture	partially aligned with the project requirements, but has notable limitations, and it doesn't fully meet the (NFRs)	Maintainability issues, rapid development, high complexity, security limitations, unscalable, high latency

Table 9 : Candidate Architectures Final Ranking

5.3. Selected System Architecture

This section describes the final architectural design selected for the Nubl platform after evaluating multiple alternatives. The chosen architecture balances security, scalability, maintainability, and development feasibility for a three-member capstone team while meeting all functional and non-functional requirements of the system.

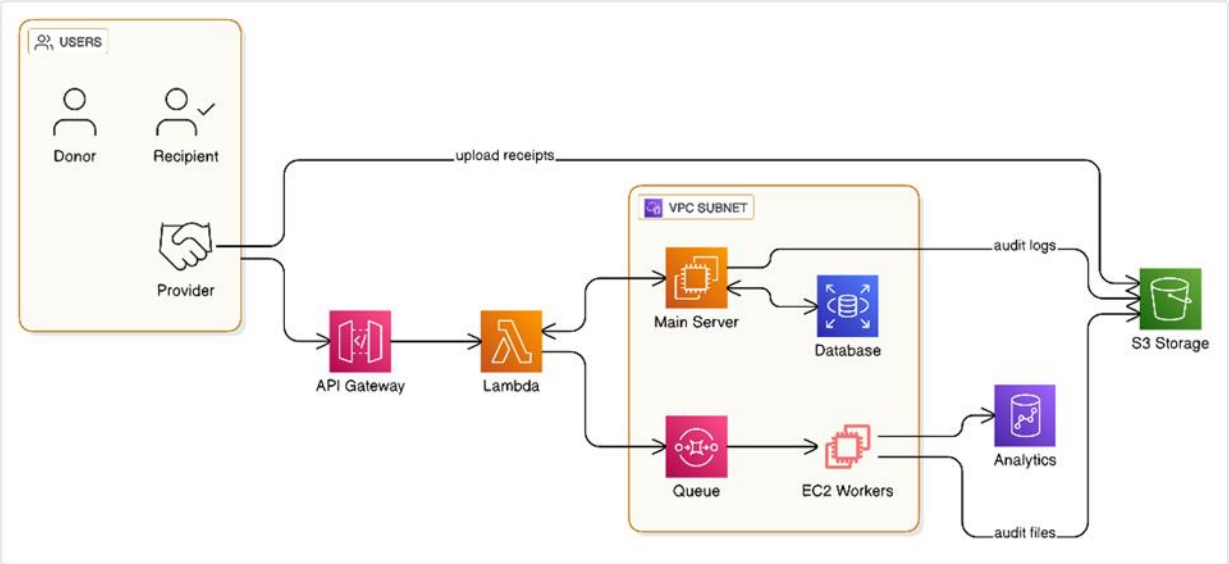


Figure 10: Our System Architecture (5.3)

5.3.1 Architecture Overview

Figure 10 illustrates the Nubl architecture split into a public entry layer and a private processing layer inside a secured VPC. Public requests are validated before reaching the core business logic. Long-running tasks are executed asynchronously using a queue-worker pattern. Reporting is generated using read-only access to prevent exposure of PII.

A) Component Responsibilities (What each component does)

API Gateway (Public Entry Layer)

- Purpose: Public entry point for web/mobile requests and routing to backend.
- Key duties: basic rate limiting, request routing, auth header forwarding.
- Connects to: Lambda (validation), Main Server (for direct routes if needed).

AWS Lambda (Validation & Pre-processing)

- Purpose: Lightweight validation before reaching core logic.
- Key duties: input validation, request normalization, token checks, reject malformed requests early.
- Connects to: Main Server.

Main Server (Core Business Logic – inside VPC)

- Purpose: Executes all core workflows (registration approval, donations, requests, QR redemption, audit logging).
- Key duties: RBAC enforcement, transaction orchestration, calling external APIs (MyFatoorah), emitting events to Queue.
- Connects to: Database, Queue, S3, Analytics (read/export), Email/Notification service (if used).

Database (Persistent Storage)

- Purpose: Stores system records and enforces data rules.
- Key duties: users/roles, requests, donations, allowances, QR tokens, audit indexes, policies.
- Connects to: Main Server (read/write), Analytics (read-only).

Queue (Asynchronous Task Buffer)

- Purpose: Decouples slow/long tasks from user requests.
- Key duties: background jobs such as email sending, report generation, audit export creation, retry jobs.
- Connects to: Main Server (producer), EC2 Workers (consumers).

EC2 Workers (Background Processing)

- Purpose: Execute queued jobs reliably.
- Key duties: retries, scheduled reports, heavy exports, reconciliation tasks.
- Connects to: Queue, Database, S3.

S3 Storage (Immutable Files & Evidence Storage)

- Purpose: Stores uploaded receipts and audit artifacts.
- Key duties: receipt image/PDF storage, report files, audit CSV/PDF, checksum files.
- Connects to: Main Server (upload/read), Workers (generate/upload), Analytics (read).

Analytics / Reporting Component

- Purpose: Produces aggregated dashboards and exports without exposing PII.
- Key duties: compute aggregated stats, generate donor transparency reports, enforce no-PII rules.
- Connects to: Database + S3 (read-only), Main Server (serves dashboards/exports).

B) Connection & Data Flow (How components interact)

Flow 1 - User Request (Normal Synchronous Request)

1. A client submits a request through the API Gateway.
2. The API Gateway forwards the request to AWS Lambda for validation and pre-processing.
3. After successful validation, AWS Lambda passes the request to the Main Server.
4. The Main Server performs the required business logic and interacts with the Database for reading or writing data.
5. The Main Server returns the processed response to the client.

Flow 2 - File Upload (Receipt / Proof of Fulfillment)

1. A client submits a file upload request through the API Gateway.
2. The request is validated by AWS Lambda and forwarded to the Main Server.
3. The Main Server stores the uploaded file in S3 Storage.
4. The Main Server records the file reference and corresponding audit entry in the Database.

Flow 3 - Background Tasks (Asynchronous Processing)

1. The Main Server publishes background jobs to the Queue for asynchronous processing.
2. EC2 Workers retrieve pending jobs from the Queue for execution.
3. EC2 Workers process the jobs and persist results to the Database and/or S3 Storage as required.

Flow 4 — Analytics & Reporting

1. The Analytics component reads aggregated data from the Database and S3 Storage using read-only access.
2. The Main Server exposes analytics dashboards and exported reports to Donor and Admin users without exposing PII.

C) Why This Satisfies Responsibilities and Connections

- Each architectural component has a clearly defined responsibility.

- All interactions among components are explicitly documented.
- Data flows demonstrate full traceability from request intake through processing, storage, and reporting.

5.3.2 Justification for the Selected Architecture

The chosen architecture was selected based on the following considerations:

A. Alignment with Sensitive Financial Requirements

The system handles donations, fund assignments, identity verification, and provider redemptions. A simple single-server design cannot guarantee sufficient security or auditing capability.

The selected architecture offers:

- A secured VPC subnet for sensitive components
- Immutable storage for receipts and audit logs
- Background workers for controlled financial processing

These features directly support the core requirements of Nubl.

B. Balanced Security Without Over-Engineering

The design avoids unnecessary complexity such as global load balancers, multi-AZ failover, or microservices.

Instead, it provides:

- API Gateway for request filtering
- Lambda for lightweight validation
- Private database communication
- Controlled access to S3 for file integrity

This meets confidentiality, integrity, and auditability NFRs without overwhelming the team.

C. Clear Separation of Responsibilities (Maintainability)

Each component handles a specific function:

- API Gateway for the public entry point
- Lambda for the validation and preprocessing
- Main Server for the core business logic
- Database for persistent storage
- Queue + Workers for the background/long-running tasks
- S3 for the files, receipts, audit logs
- Analytics for the reporting

This separation reduces coupling and improves maintainability.

D. Scalable Enough for MVP Needs

The queue and worker model supports future growth without requiring microservices.

Examples:

- Additional workers can be added as load increases
- File storage scales naturally with S3
- Lambda handles concurrent request bursts

The system retains elasticity without high operational overhead.

E. Feasible for the Team's Skills and Timeline

Unlike a fully serverless or enterprise architecture, this design allows the team to:

- Use one centralized backend (Laravel/Spring/Node)
- Keep AWS usage limited to essential services
- Focus on implementing domain logic rather than infrastructure

This ensures the architecture is realistic to build within a semester while still reflecting professional engineering standards.

5.4. A More Detailed Design

5.4.1. Entity Relation Diagram - ERD

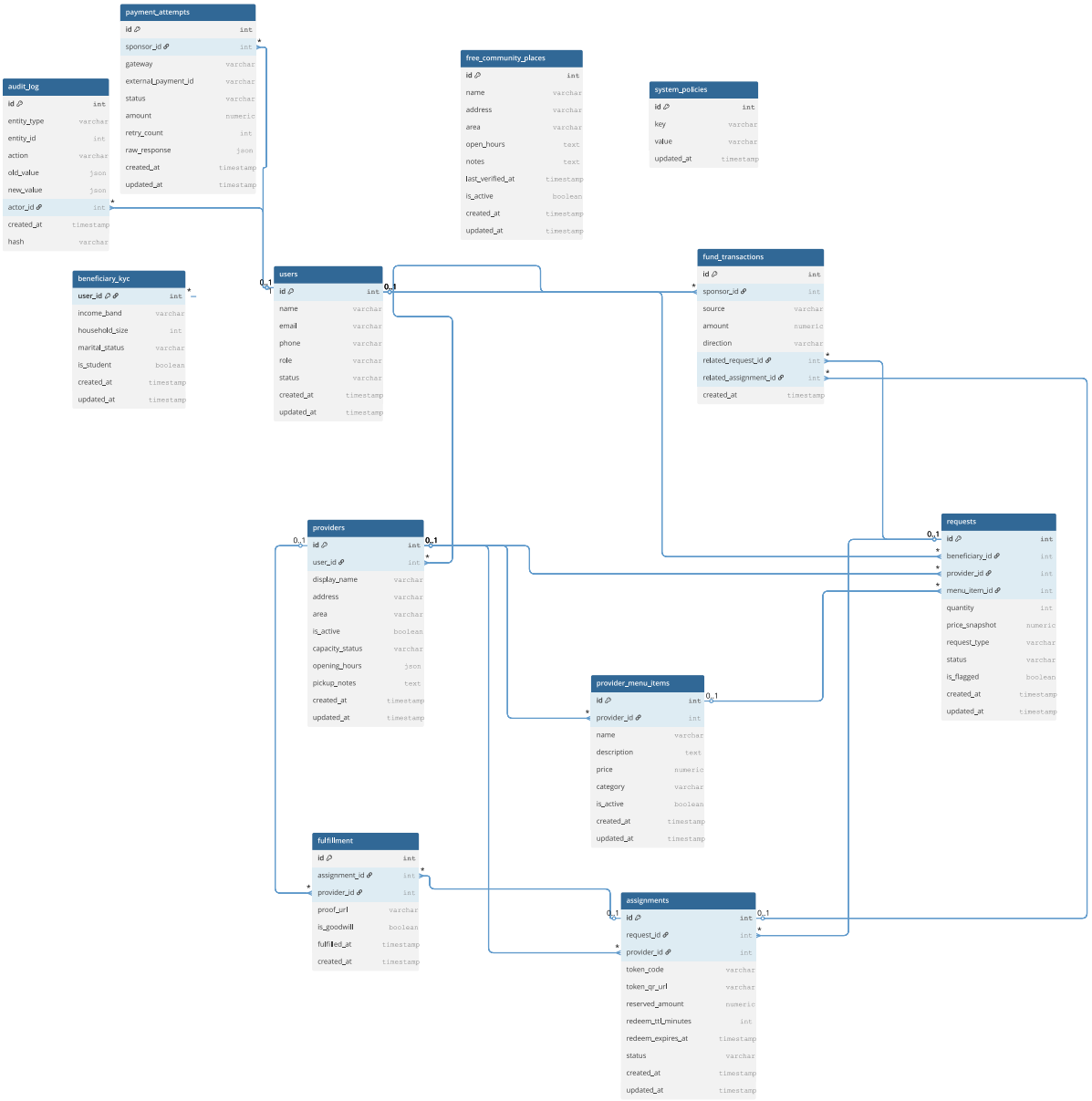


Figure 11: Entity Relation Diagram – ERD (5.4.1)

The Entity-Relationship Diagram (ERD) above represents the core data model of the Nubl Platform. The system manages beneficiaries, providers, fund pools, and digital vouchers. It supports submitting requests, assigning funds, fulfilling purchases, and recording financial and

operational activity. The platform ensures transparency, traceability, and efficient distribution of assistance.

5.4.2. Class Diagram

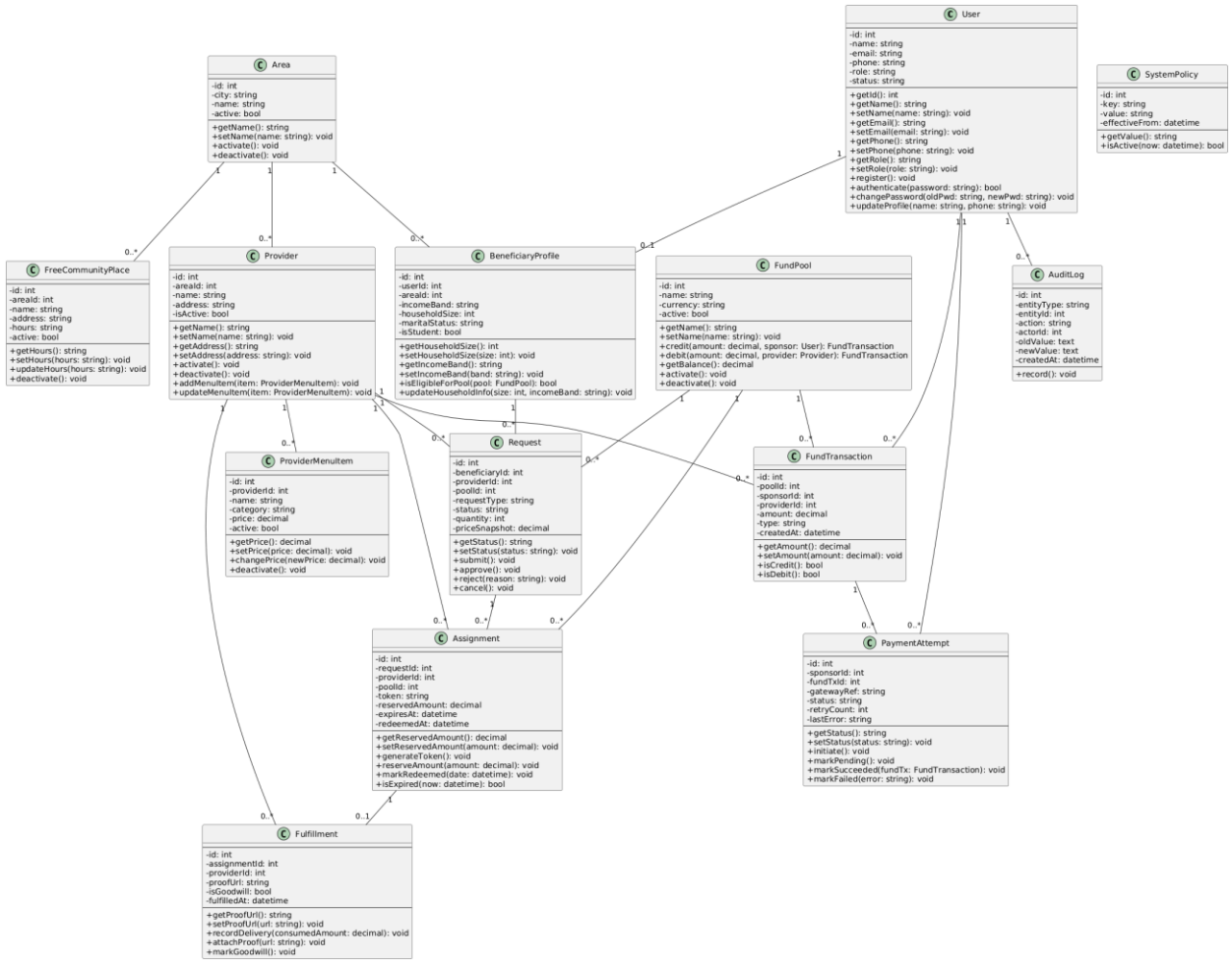


Figure 12: Class Diagram (5.4.2)

The above class diagram illustrates the static structure of the Nubl Platform. It defines the main domain entities, their private attributes, and public operations, as well as the associations between them. The model represents the core concepts of the system, including users, beneficiaries, providers, fund pools, financial transactions, and the request assignment fulfillment workflow. Encapsulation is applied through private attributes and public getter/setter operations, ensuring proper object-oriented design. The multiplicities between classes reflect real-world constraints, such as one user having at most one beneficiary profile, a fund pool supporting multiple requests, and each assignment having at most one fulfillment. This diagram provides the structural foundation for the system's behavior and API design.

5.4.3. Sequence Diagram- User Registration & Approval

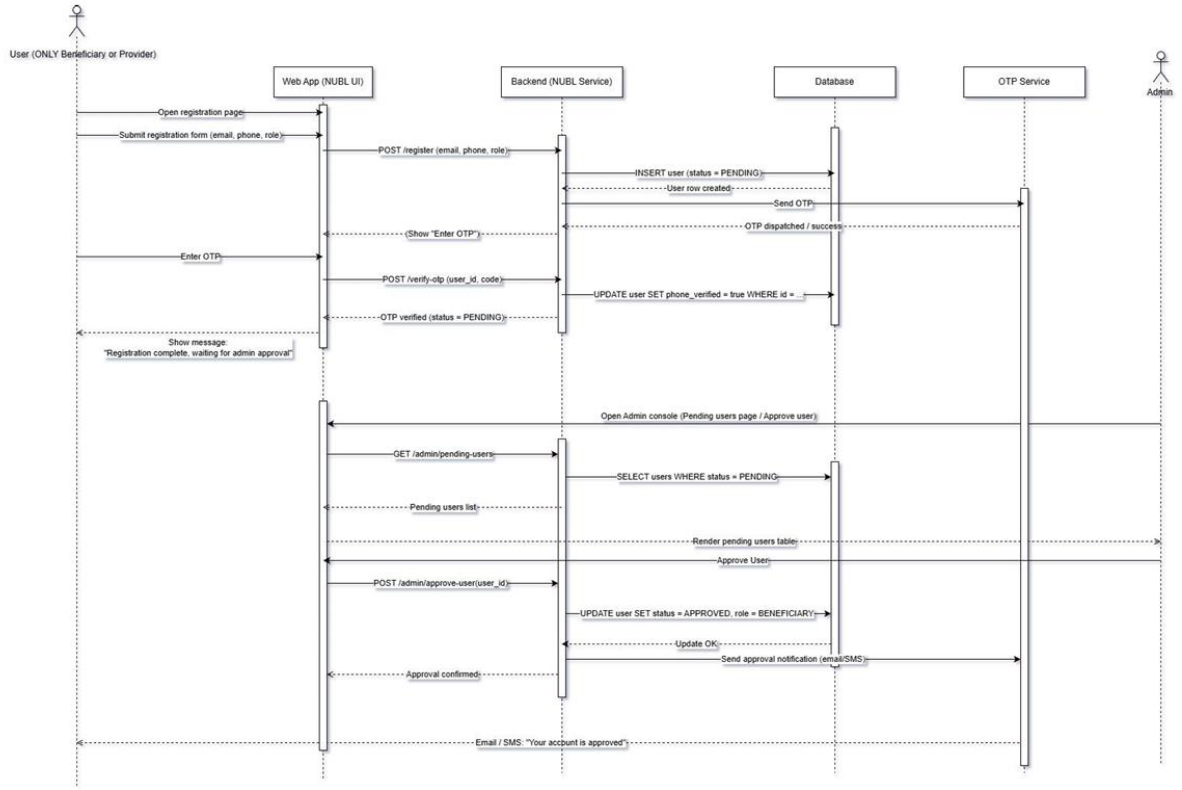


Figure 13: Sequence Diagram – User Registration & Approval (5.4.3)

- This diagram models the registration flow for Beneficiary and Provider accounts (FR-1.1-1.5, FR-2.1. FR-14.1).
- The User interacts with the NUBL Web App to submit registration data, which the Backend persists in the database with status PENDING and triggers an OTP send through the external OTP Service.
- After the user enters the OTP, the Backend verifies it and keeps the account in PENDING state until an Administrator reviews it.
- The Admin opens the Pending Users page, which calls an API to list users with status = PENDING. When the Admin approves a user, the Backend updates the record to APPROVED and sends an email/SMS notification.
- The diagram highlights the separation between UI, business logic, database operations, and external notification service.

5.4.4. Sequence Diagram- Beneficiary Request Creation & Allowance Check

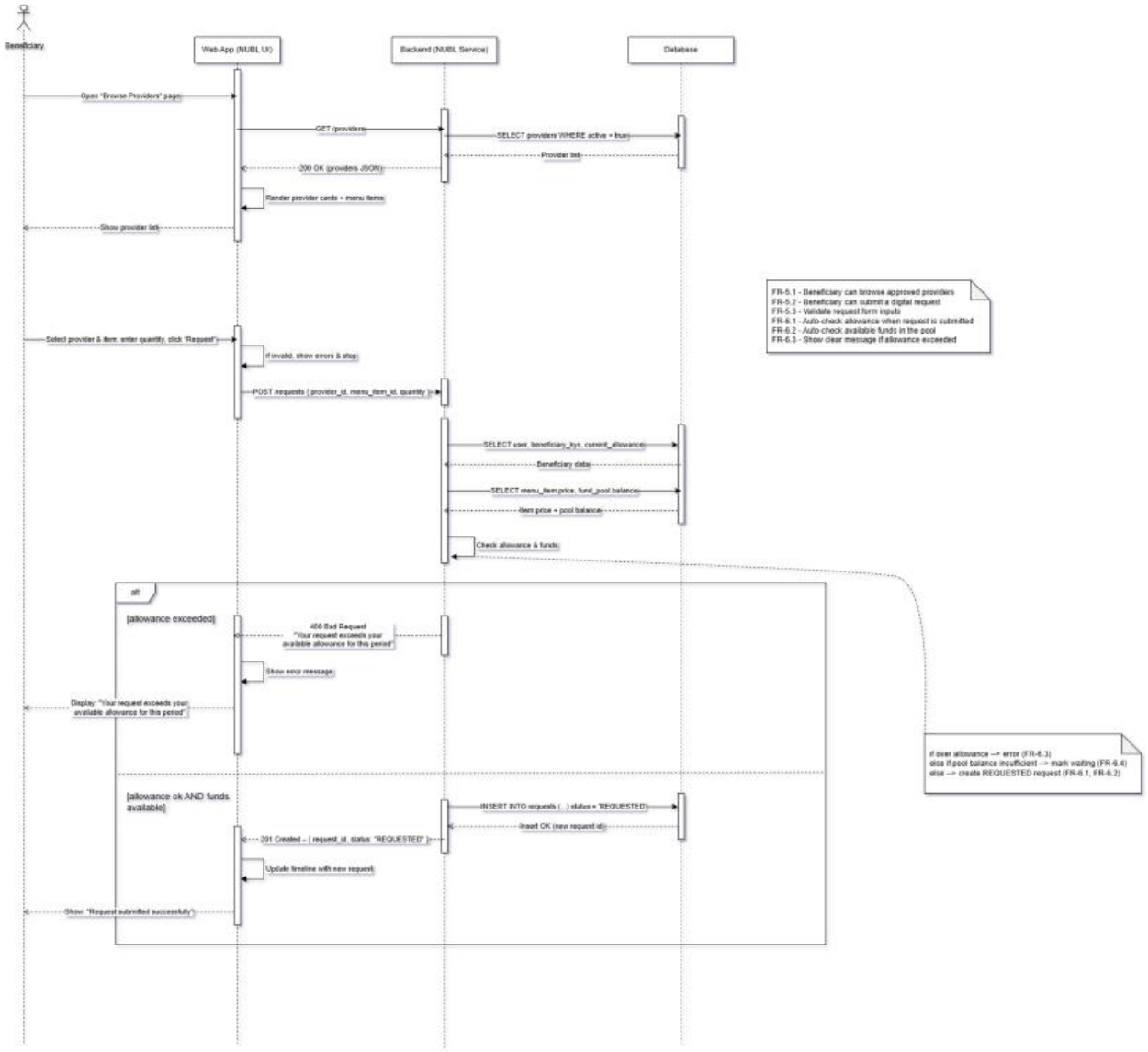


Figure 14: Sequence Diagram – Beneficiary Request Creation & Allowance Check (5.4.4)

- This diagram covers the creation of a grocery request by a Beneficiary (FR-5.1–5.3, FR-6.1–6.3).
- The Beneficiary browses approved Providers, selects an item and quantity, and submits the request through the Web App. The Backend validates input and retrieves both the user's allowance and the pool balance. An alt fragment shows two outcomes:
    - If allowance or funds are insufficient, the Backend returns a 400 error and the Web App displays an informative message.

- If both checks pass, the Backend inserts a new REQUESTED row in the requests table, and the Web App updates the timeline to show that the request was submitted successfully.

5.4.5. Sequence Diagram - Admin configures system-wide allowance (FR-17.1)

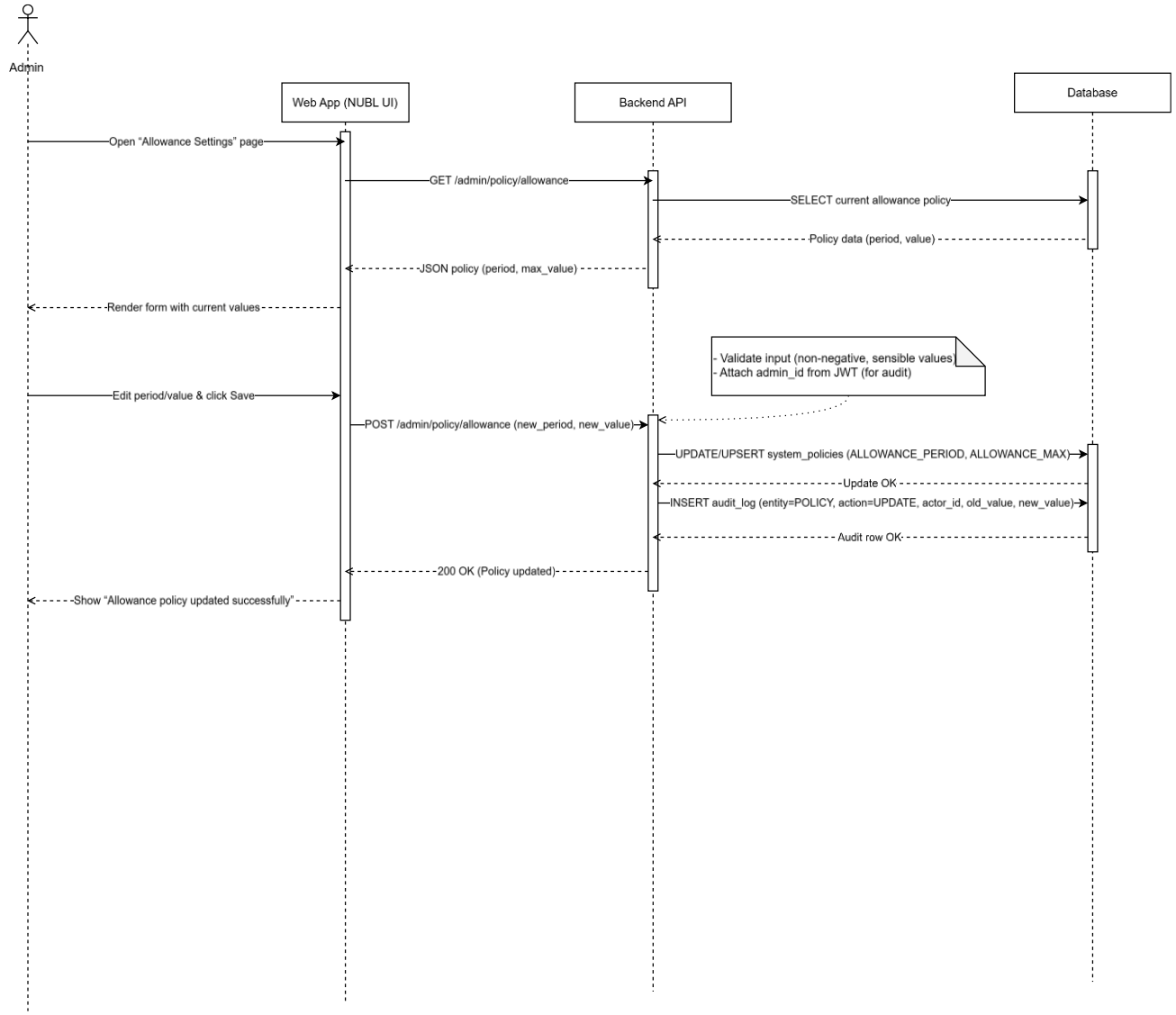


Figure 15: Sequence Diagram - Admin configures system-wide allowance (5.4.5)

This sequence diagram shows how an Admin updates the global allowance policy. The Admin opens the Allowance Settings page, the Web App calls the Backend API to fetch current values from the database, and the form is rendered. When the Admin submits new allowance values, the Backend API validates them, updates the system_policies table, writes an audit_log entry, and returns a success response. Future Beneficiary requests are validated against these updated allowance values.

5.4.6. Activity Diagram- Beneficiary Request & Fulfillment

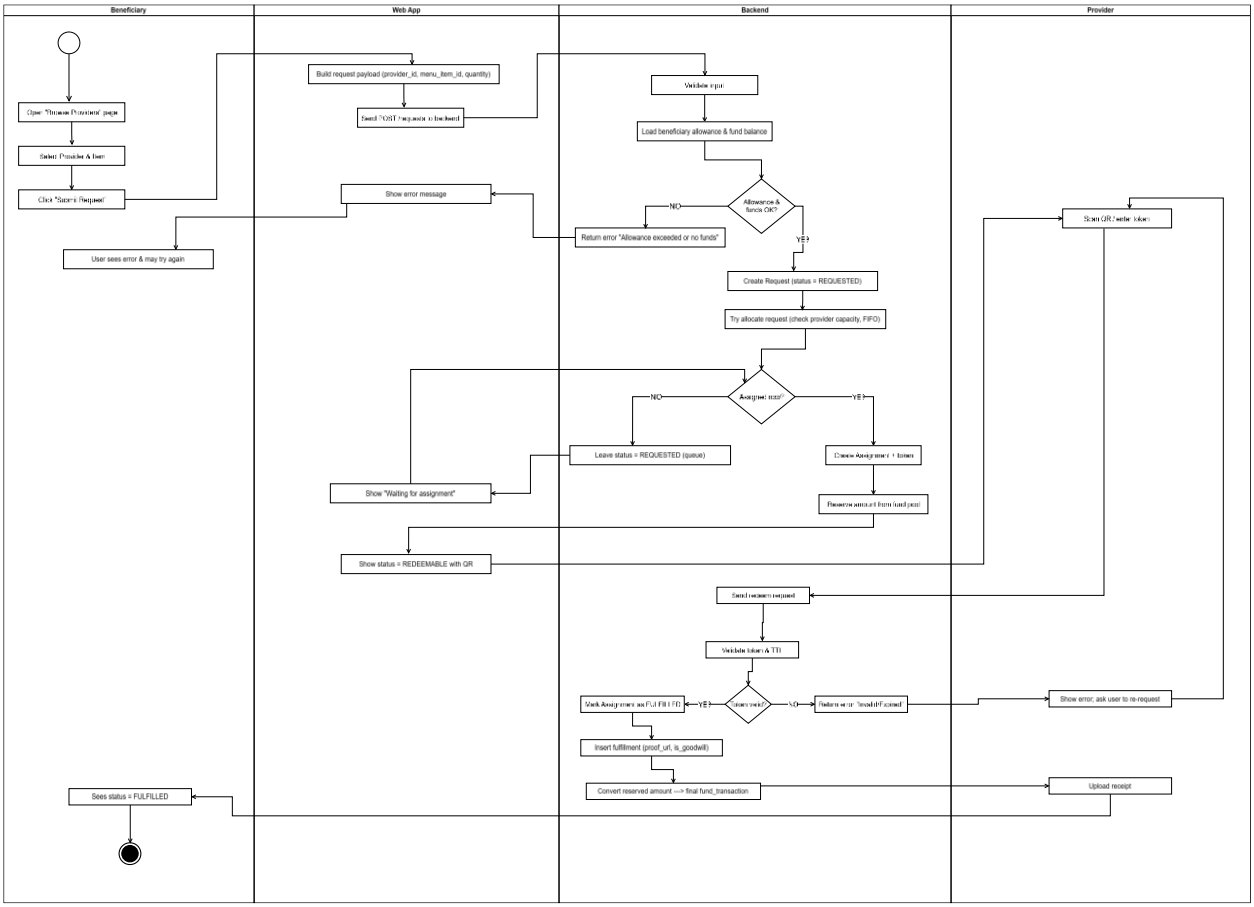
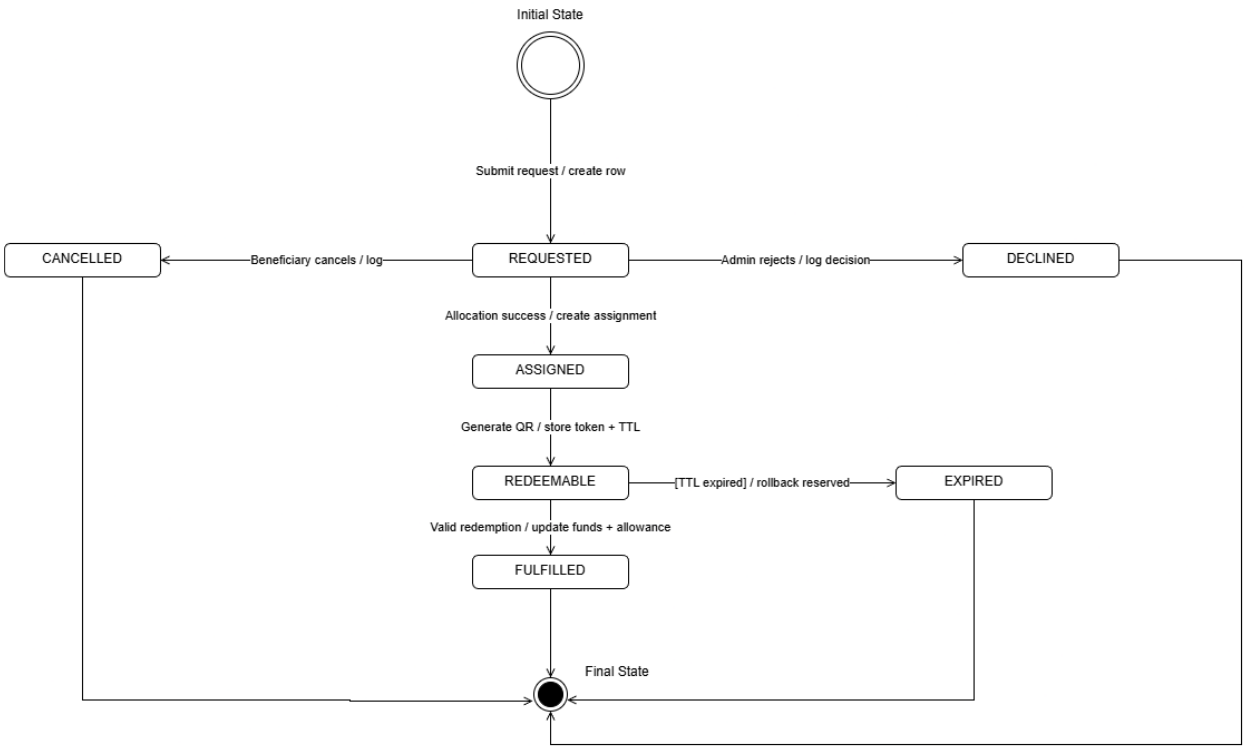


Figure 16: Activity Diagram- Beneficiary Request & Fulfillment (5.4.6)

- This activity diagram describes the end-to-end flow from browsing providers to final fulfillment (FR-5. , FR-6. , FR-8. ,FR-11.). With separate responsibilities between Beneficiary, Web App, Backend, and Provider.
- The diagram shows request submission, validation, allowance and fund checks, conditional allocation, QR/token generation, provider redemption, error handling for invalid/expired tokens, fulfillment recording, and fund transaction updates. It acts as a bridge between user stories and the more detailed sequence/state diagrams.

5.4.7. State Diagram - Request Lifecycle



- This state machine defines all allowed states of a request record and the transitions between them.
- A request starts in REQUESTED after submission. It may be cancelled by the Beneficiary (CANCELLED), rejected by an Admin (DECLINED), or assigned to a Provider (ASSIGNED). Once an assignment exists, a QR token is generated and the request becomes REDEEMABLE.
- From REDEEMABLE, the request either moves to FULFILLED after a valid redemption (with atomic fund and allowance updates) or to EXPIRED when the TTL passes and the reserved amount is rolled back. All four terminal states lead to the final state, guaranteeing every request ends in a well-defined status for auditing and reporting.

5.4.8. Sequence Diagram – Payment

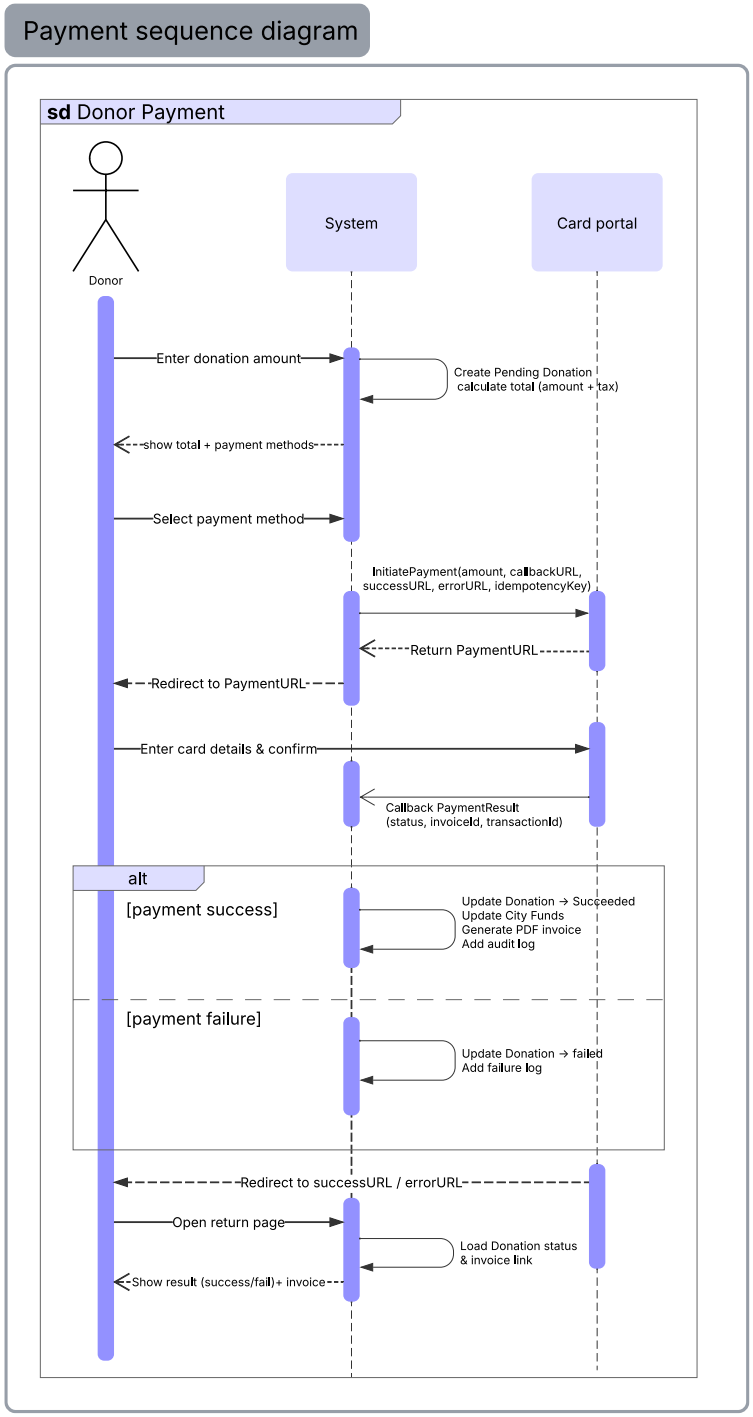


Figure 17: Sequence Diagram – Payment (5.4.8)

- This sequence diagram shows how the donor makes a payment, starting from entering the donation amount until the payment result is returned and processed by the system.

- It explains what happens when the payment is successful or fails, including updating the donation status, handling funds, and showing the final result to the donor.

5.4.9. Activity Diagram – Payment

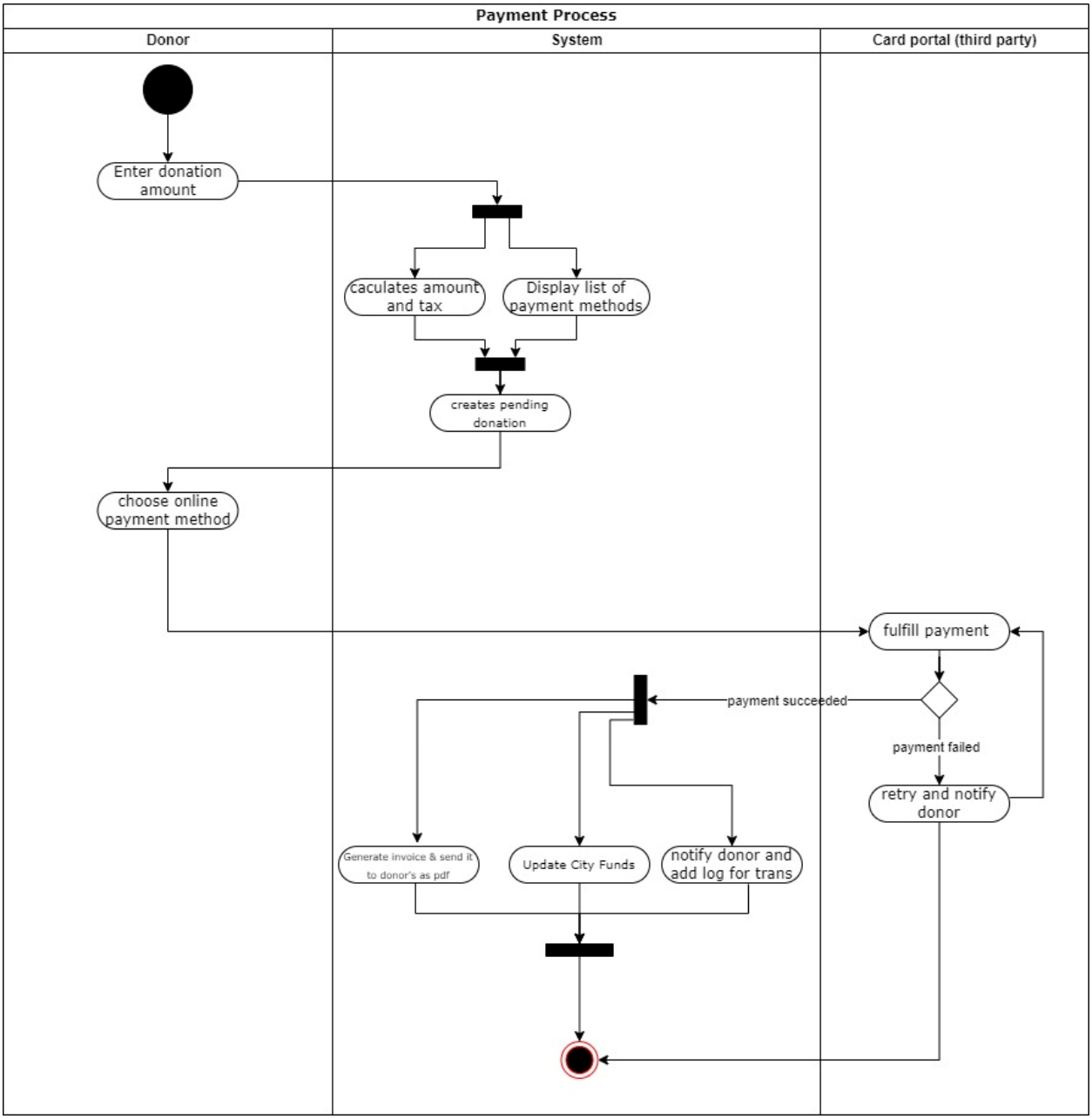


Figure 18: Activity Diagram – Payment (5.4.9)

- This activity diagram shows the steps of the payment process starting from the donor entering the donation amount, choosing the online payment method, and sending the payment request to the card portal.

- It explains how the system handles the payment result by either completing the payment, updating the city fund and generating an invoice if successful, or recording the failure and notifying the donor if the payment is not completed.

5.5.1. State Diagram – Payment Process State

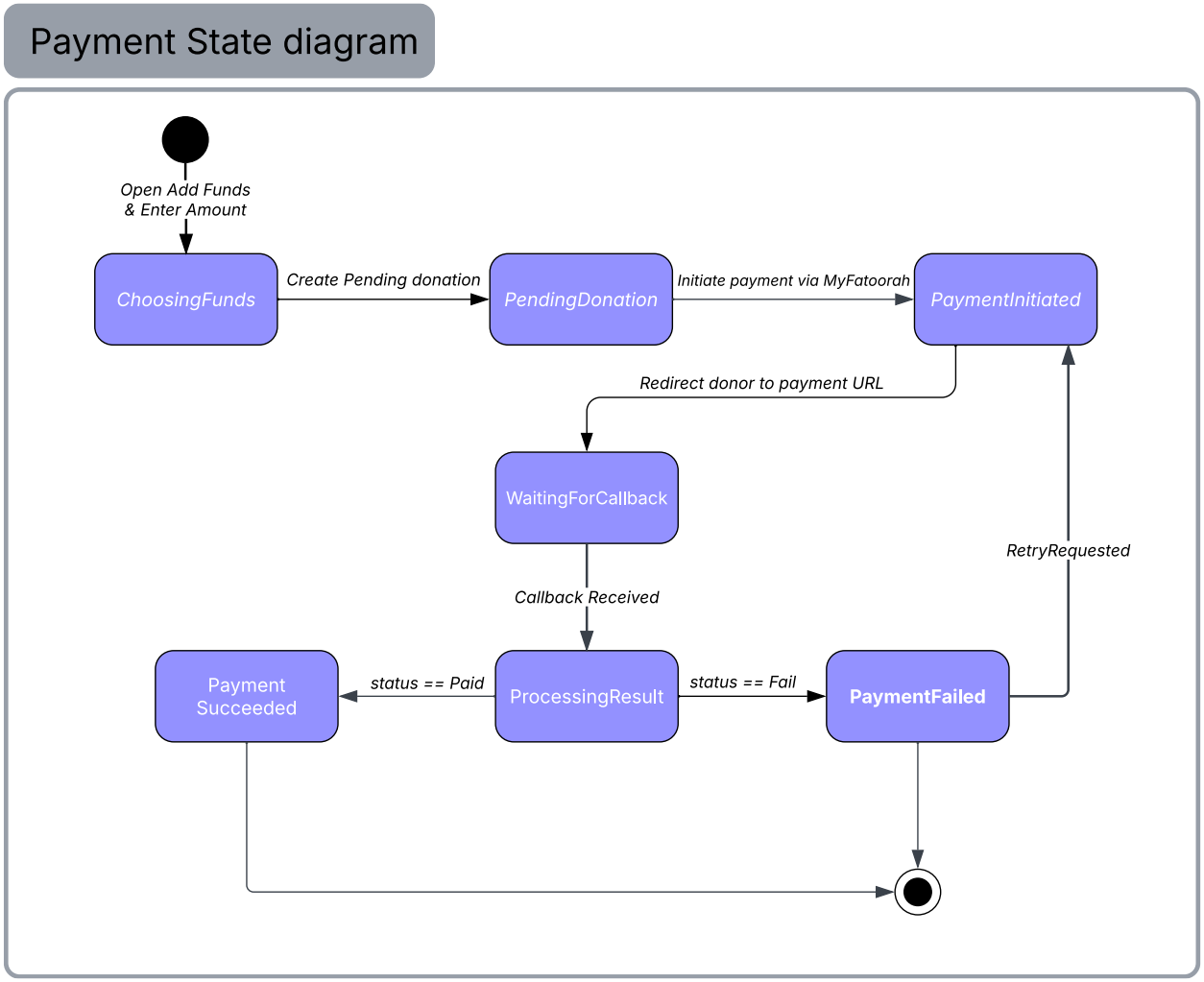


Figure 19: State Diagram – Payment Process State (5.5.1)

- This state diagram shows how a donation payment moves between different states, starting from entering the amount, then pending and processing, until it ends as either a successful payment or a failed payment with the option to retry.

5.5.2. Sequence Diagram – Donor Impact Dashboard

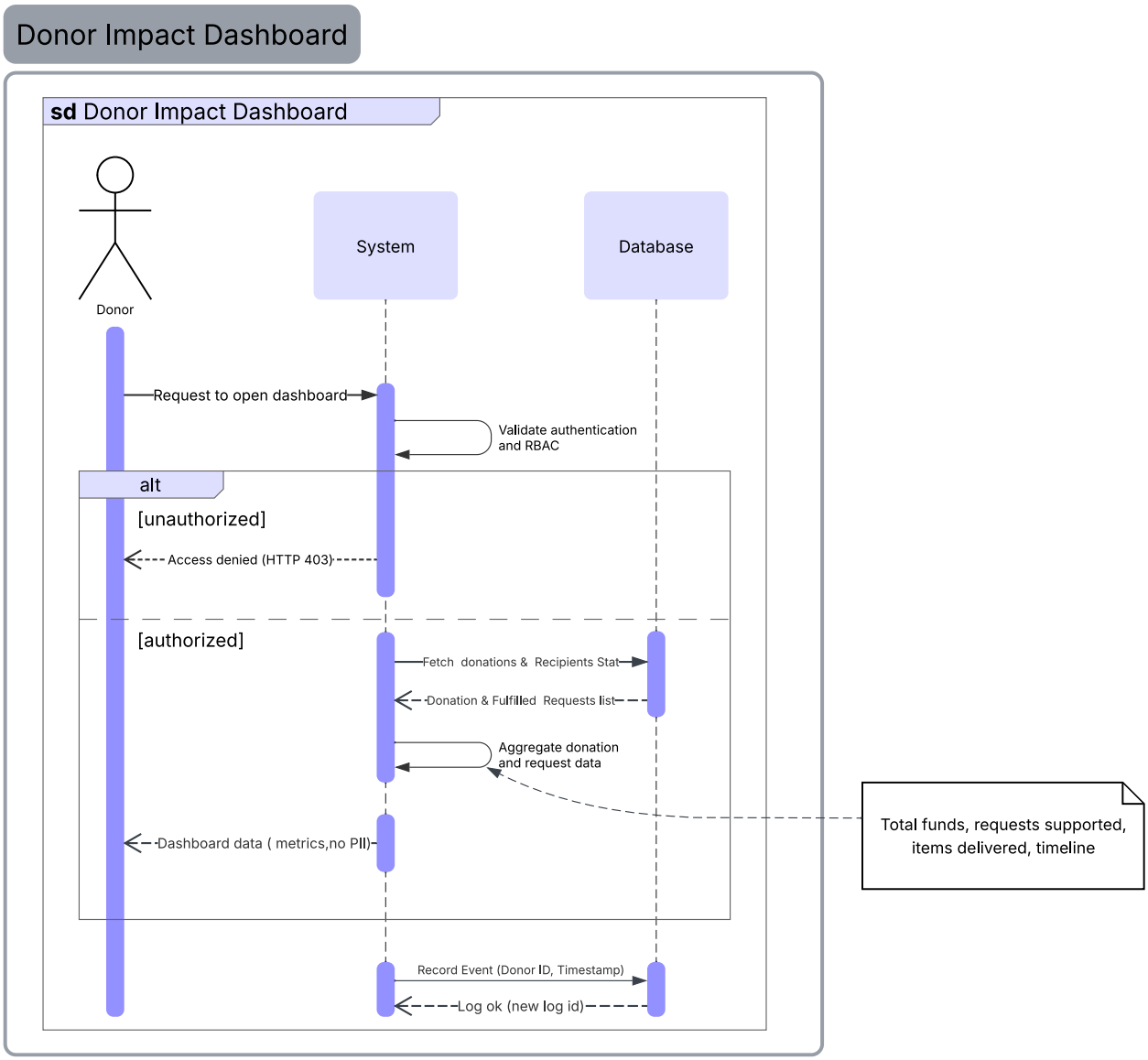


Figure 20: Sequence Diagram – Donor Impact Dashboard (5.5.2)

- This sequence diagram shows how the donor opens the impact dashboard and how the system first checks authentication and access permissions.
- It explains how the system retrieves donation and request data from the database, aggregates the information, and prepares dashboard metrics.
- The diagram shows that only summarized and non-personal data (such as total funds, supported requests, and delivered items) is displayed to the donor to protect privacy.

5.5.3. Activity Diagram – User Registration & Approval

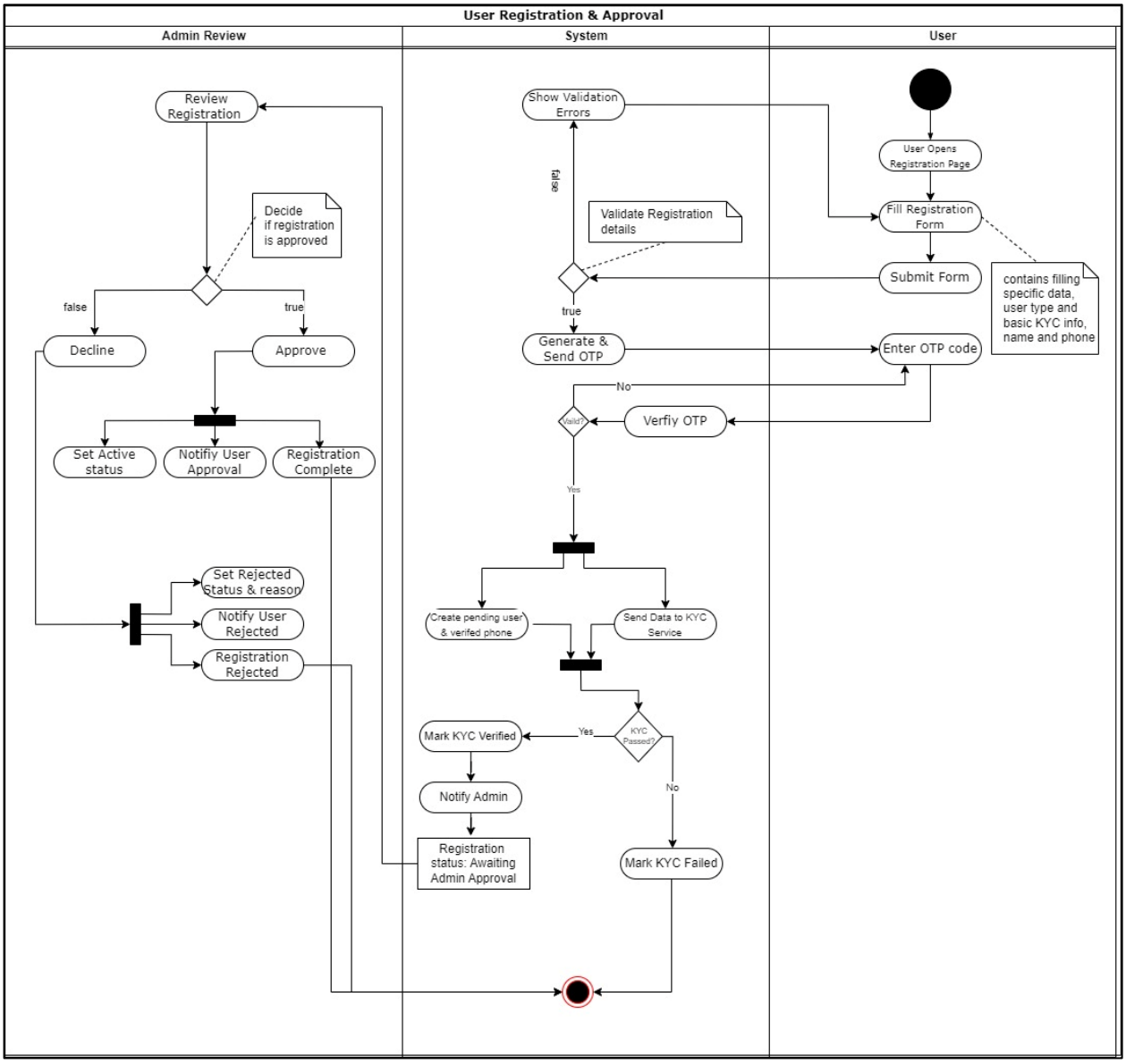


Figure 21: Activity Diagram – User Registration & Approval (5.5.3)

- This activity diagram shows the steps of user registration, starting from filling the registration form and verifying the phone number using an OTP.
- It explains how the system validates the user data, performs basic KYC checks, and marks the registration as pending until admin review.
- The diagram also shows the admin decision process, where the admin either approves the user and activates the account or rejects the registration and notifies the user.

5.5.4. Sequence Diagram – QR Redemption and Atomic Balance Update

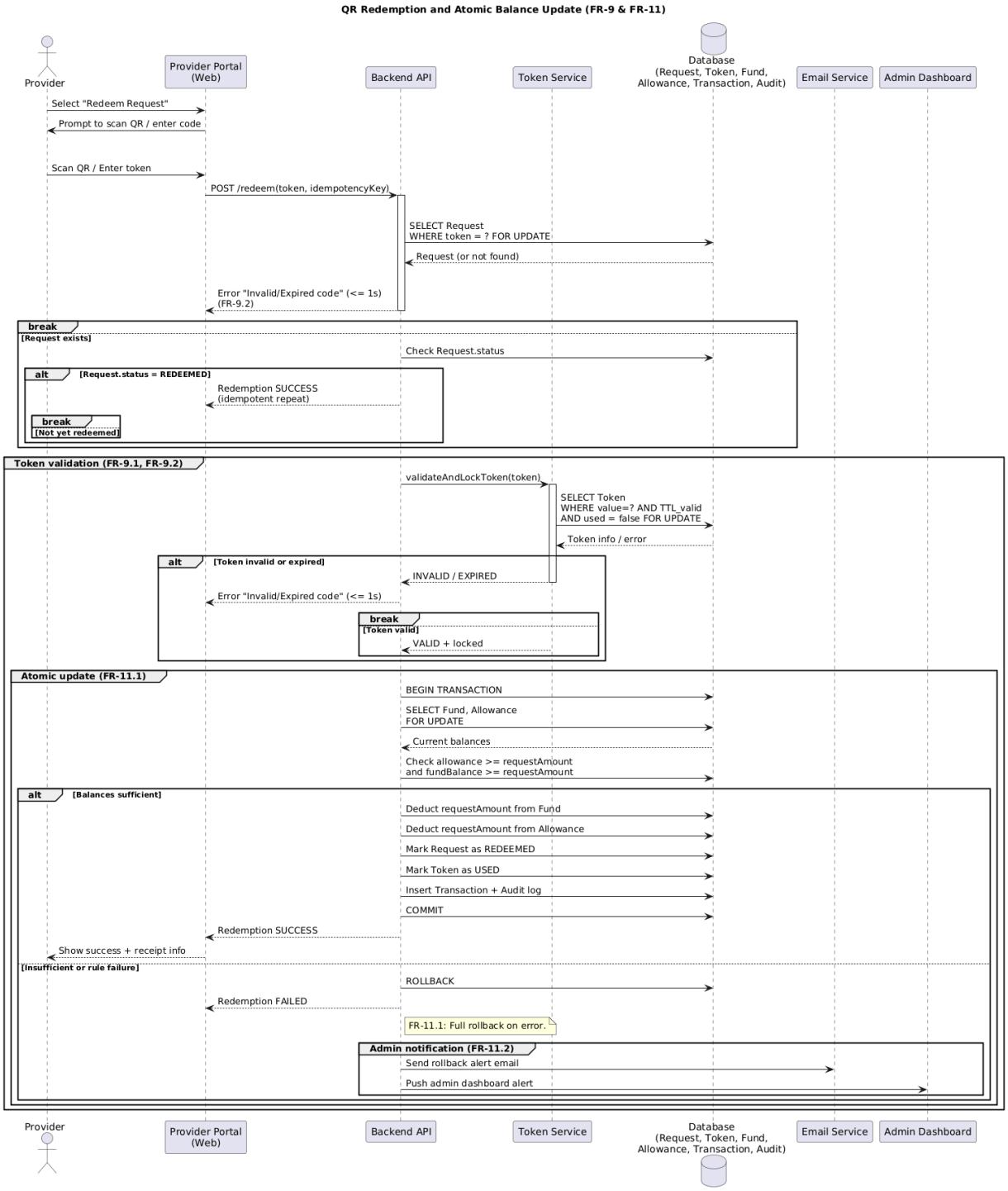


Figure 22: Sequence Diagram – QR Redemption and Atomic Balance Update (5.5.4)

- This sequence diagram describes the provider redemption workflow, where a request is redeemed by scanning a QR code or submitting a redemption token through the provider portal.

- It illustrates the system's token validation process, including verification of token validity, expiration status, and prior usage, as well as token locking to prevent duplicate or replayed redemptions.
- The diagram demonstrates the atomic transaction mechanism used to update system balances, where the requested amount is deducted simultaneously from the city fund and the beneficiary allowance.
- It also models both successful and failure execution paths, showing how the system performs transaction rollback and triggers administrative notifications in the event of errors or redemption failure.

5.5.5. Activity Diagram – QR Redemption and Atomic Balance Update

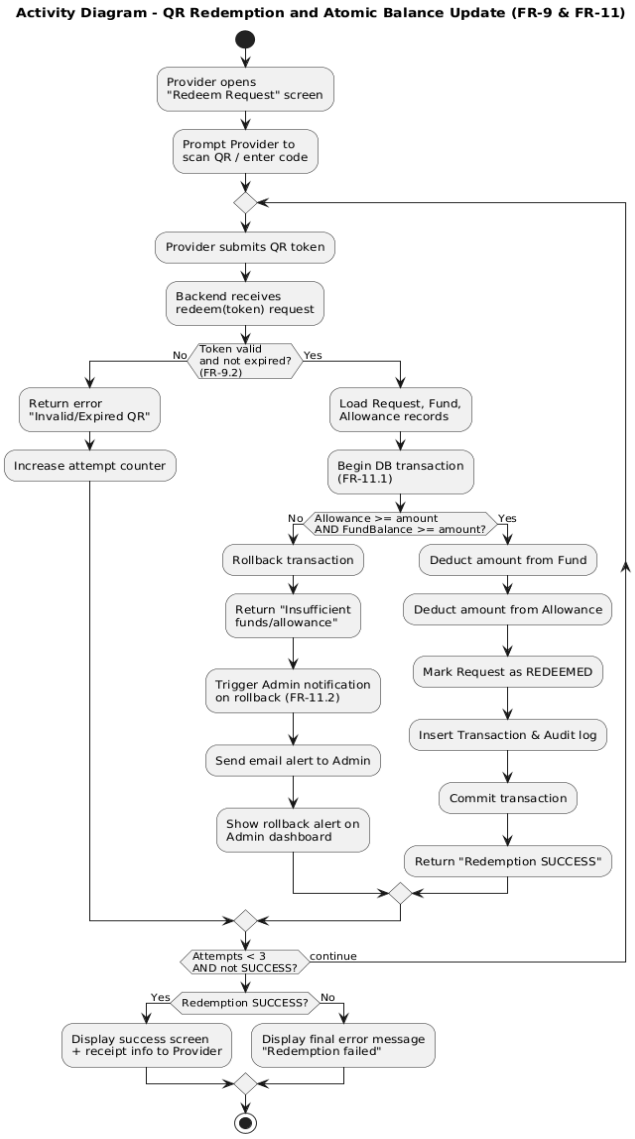


Figure 23: Activity Diagram – QR Redemption and Atomic Balance Update (5.5.5)

- This activity diagram shows the steps a provider follows to redeem a request using a QR code, including token validation and checking available funds and allowance.
- It explains how the system completes the redemption by updating balances in a single transaction or rolls back the process and notifies the admin if an error occurs.

5.5.6. State Diagram – QR Redemption and Atomic Balance Update

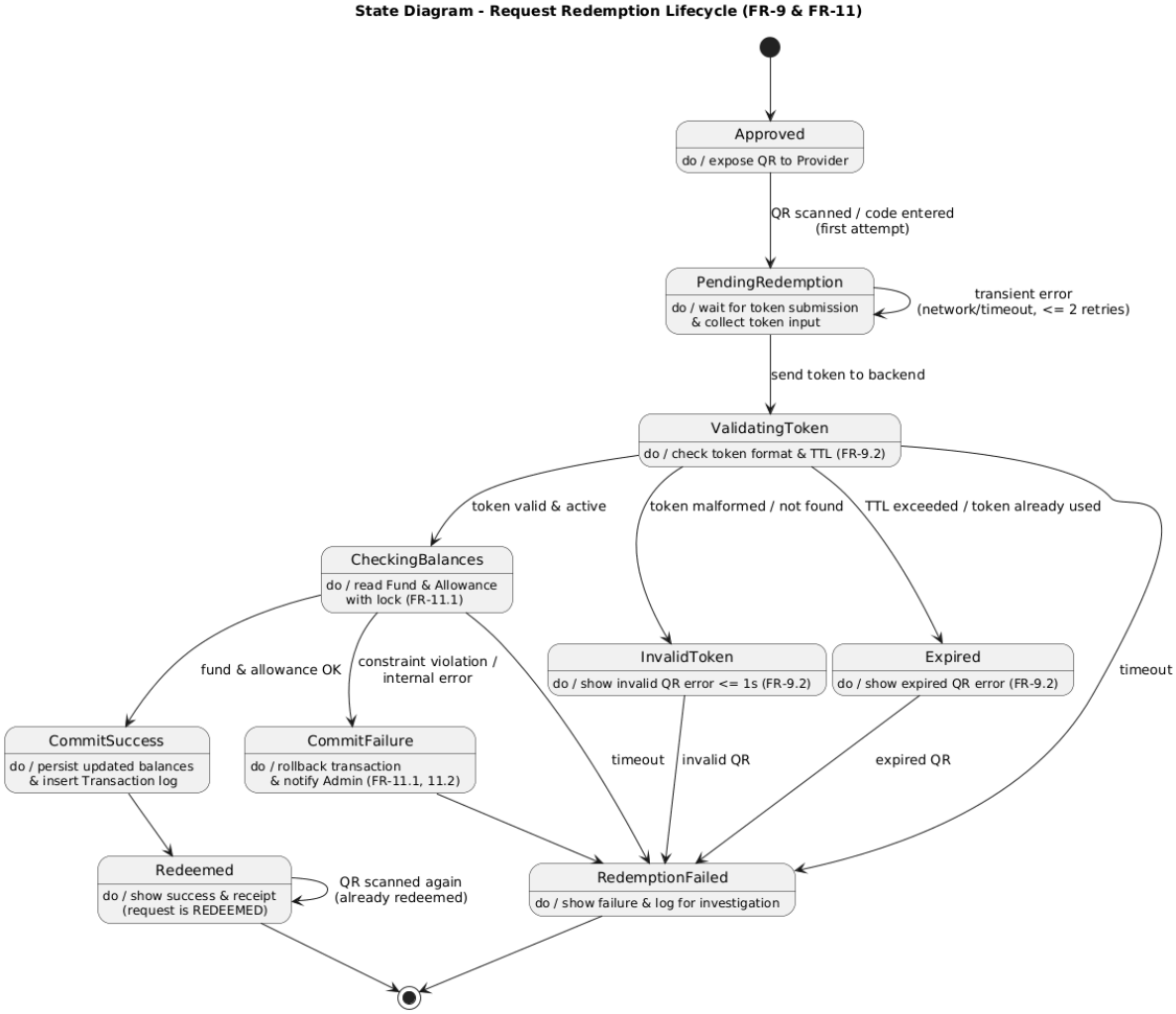
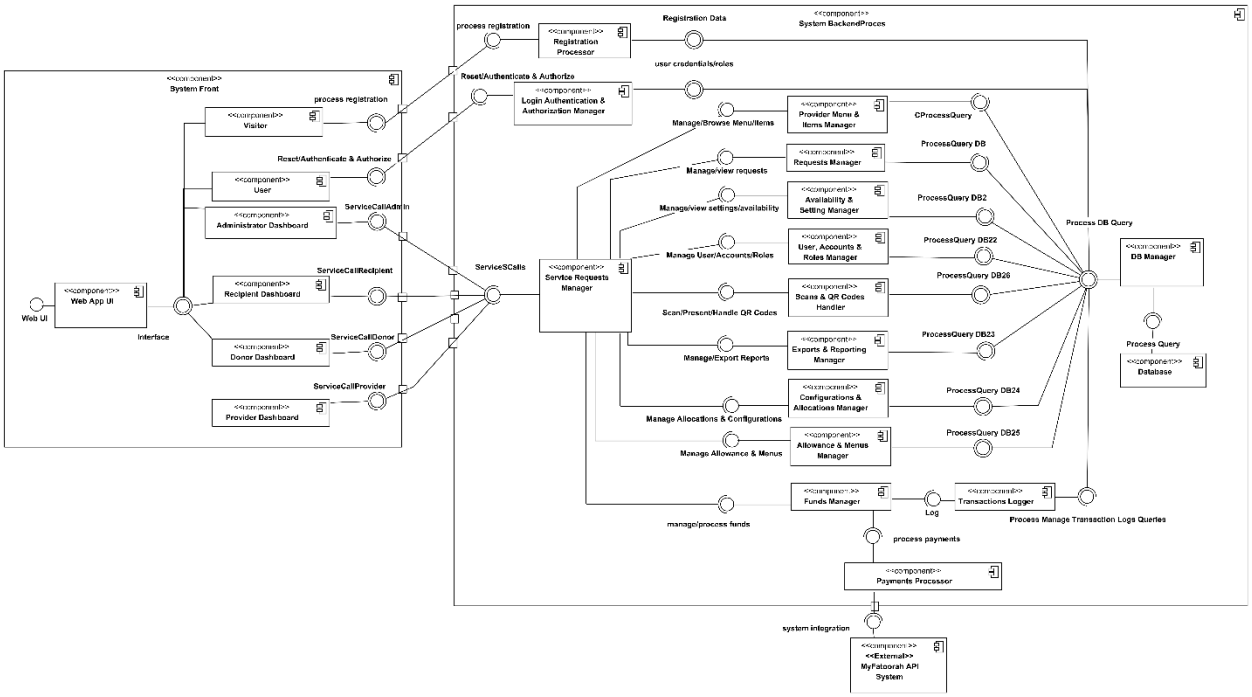


Figure 24: State Diagram – QR Redemption and Atomic Balance Update (5.5.6)

- This state diagram models the complete lifecycle of a request redemption, starting from the approved state and progressing through token validation, balance checking, and final redemption or failure states.
- It clearly defines valid and invalid state transitions, including handling of expired or malformed tokens, retry limits, and timeout conditions, in accordance with FR-9 token validation requirements.

- The diagram also enforces transactional integrity by showing atomic commit and rollback paths (FR-11), ensuring that fund and allowance updates either complete successfully or are safely reverted with administrative notification.

5.5.6. Component Diagram



- **User interaction and request initiation**
  The system starts at the Web App UI, where different user roles (Visitor, Recipient, Donor, Provider, Administrator) interact through their respective dashboards. User accessions (such as registration, login, requests, donations, or scanning QR codes) are forwarded as service calls to the backend through well-defined interfaces.
- **Business logic orchestration in the backend**
  All incoming requests are handled by the Service Request Manager, which coordinates core backend components such as authentication and authorization, user and role management, availability settings, QR code validation, reporting, allocations, and funds management. Each component is responsible for a specific business function, ensuring separation of concerns and modularity.
- **Data processing and external system integration**
  Backend components interact with the DB Manager to process and retrieve data through structured database queries, while financial operations are logged and tracked via the transactions logger. Payment-related actions are securely processed through the Payment Processor, which integrates with the external MyFatoorah API, completing the system's end-to-end workflow.

6. Future work (To be Done in Capstone 2)

In Capstone II, we will implement the full NUBL system as a real working web application. We will start by setting up the project structure, choosing the final tech stack, and preparing the development environment for the team. Then we will build the database tables and relationships (users, requests, fund transactions, providers, receipts, and audit logs). After that, we will connect the database to the backend APIs and make sure the APIs are secure and well organized. We will also create the main pages for each role (neighbor/beneficiary, donor, provider, and admin) so every user has a clear dashboard and a simple navigation. Finally, we will add login, registration, and role-based access control to make sure every user can only see and do what is allowed for their role.

Next, we will develop the main workflow step by step and test it after each stage. A beneficiary will be able to submit a request by selecting items, writing notes, and sending it for review. The system will check the allowance rules automatically to avoid unfair requests, and it will also handle special cases that need manual review. The admin will have a dashboard to view new requests, approve or reject them, and track their status. After approval, the provider will receive the request details needed for fulfillment, and the system will generate a secure one-time QR code. The provider will redeem the QR code when delivering the groceries or meal, and then upload a receipt as proof of fulfillment. In the same phase, we will integrate MyFatoorah so donors can donate safely to the city fund, and we will make sure the fund balance and transaction history update correctly after each payment.

Finally, we will improve the system quality and prepare it for a real pilot. We will add stronger validations and controls such as request status rules, QR expiration time, and limits to stop duplicate redemptions or wrong uploads. We will also add logging and audit trails so we can track important actions like approvals, payments, and redemptions, which helps with transparency. After that, we will focus on testing, including unit tests and integration tests for critical parts like payment handling, allowance checks, and QR code validation. We will run user-based scenarios to test the full flow from donation to fulfillment and fix any bugs or usability issues. At the end, we will deploy the system online, run a small trial in one city, collect feedback from users, and improve the interface, performance, and overall user experience.

7. KPI 4.1 – Professional and Ethical Standards

7.1 Ethical Issues and Professional Code of Conduct

The NUBL Platform handles sensitive personal, financial, and social data related to food assistance. The main ethical issues identified in this project include protection of beneficiary privacy, prevention of discrimination, fair allocation of funds, and responsible use of administrative authority. These issues are addressed in alignment with the ACM/IEEE Software Engineering Code of Ethics, particularly principles related to public interest, confidentiality, fairness, and accountability.

7.2 Ethical Case Studies

Case Study 1: Privacy vs Transparency

key ethical challenge is balancing donor transparency with beneficiary privacy. While donors expect visibility into how funds are used, exposing individual request details could harm beneficiary dignity. The design decision was to provide donors with aggregated statistics only, ensuring transparency without revealing Personally Identifiable Information (PII).

Case Study 2: Administrative Authority and Misuse Risk

Administrators have elevated privileges to approve users and override allowance limits in exceptional cases. This creates a risk of misuse of authority. To mitigate this, all administrative actions are logged with timestamps and identifiers, ensuring accountability and traceability.

7.3 Personal Ethical Reflection

During the design of NUBL, the team learned that ethical software design often requires trade-offs between efficiency and human dignity. We deliberately chose privacy-preserving and KYC-light approaches even though stricter verification could reduce fraud, because protecting vulnerable users from unnecessary exposure was prioritized over absolute control.

7.4 Regulatory Laws and Ethical Guidelines

The project observes relevant legal and ethical standards, including the Saudi Personal Data Protection Law (PDPL), GDPR privacy principles, OWASP security guidelines, and the ACM/IEEE Software Engineering Code of Ethics.

8. KPI 4.2 – Global, Economic, Environmental and Societal Impact

8.1 Global markets and communities impact

Nubl addresses a global humanitarian pattern such as improving aid delivery through digital workflows while protecting beneficiary dignity. While many large-scale platforms exist internationally, Nubl targets city-level and hyper-local communities enabling local supermarkets and donors to participate in structured assistance without exposing beneficiaries. This model can be adapted to other cities and communities with similar cultural needs, particularly where privacy and social dignity are essential for adoption.

8.2 Economic benefits and costs - high-level assessment

Benefits

- Reduced operational overhead compared to manual vouchers (less staff time, fewer disputes).
- Better financial accountability through audit logs and receipt proof.
- Faster fulfillment (less waiting time) and better donor confidence, increasing donations.

Costs

- Payment gateway fees + service charges.
- SMS/OTP delivery costs.

- Hosting + storage (especially receipt images).
- Admin operations (review, provider onboarding, audits).

Category	*Benefits*	*Costs*
Donors	verified impact, trust	gateway fees
Beneficiaries	faster access, dignity	Onboarding or verification friction
Providers	clearer workflow, less fraud	training and dashboard usage
Admin/Charity	auditability, reporting	operations and monitoring

Table 10 : Economic Benefits and Costs Assessment

8.3 Environmental impact

Compared to paper-based vouchers and repeated physical travel, NUBL can reduce printing and administrative paperwork. However, it introduces compute/storage impact (servers, databases, receipt uploads). To reduce environmental footprint, NUBL adopts:

- Short retention for sensitive records (reduces storage) (you already state retention control),
- By limiting file size and compressing receipts,
- By caching and efficient queries to reduce computing load,
- Scheduled cleanup of expired tokens and unused files.

9. Conclusion

In Capstone I, we worked on planning and designing the Nubl platform. The main goal of our project is to support people with groceries and meals in a respectful way, while keeping their privacy and dignity. We explained the main concept of the system and how it connects the community in a simple and organized way. The platform brings together donors who contribute money, beneficiaries who request support, providers who deliver the groceries or meals, and admins who manage approvals and control the process. We focused on making the support process fair and clear, so the help reaches the right people without exposing personal information.

During this phase, we completed the important analysis work that must be done before implementation. We studied the problem, identified the stakeholders, and wrote the functional and non-functional requirements in a structured way. We also defined the project scope as an MVP for one city to keep the plan realistic and achievable. In addition, we supported our decisions with a literature review related to digital humanitarian aid, privacy-by-design, and dignity-centered assistance. This research helped us understand best practices and risks, and it guided us to design the system with strong privacy rules and better user trust.

Finally, we have prepared a strong design foundation that will be used directly in Capstone II. We created the system architecture idea, designed the database model (ERD), and prepared clear UML diagrams such as use case, sequence, activity, and state diagrams to show the system workflow step by step. These diagrams explain how requests move from submission to approval and then fulfillment, and they also show how payments and QR redemption will work. Overall, Capstone I gave us a clear and organized plan, and it reduced risks for the next phase. In Capstone II, we will use this design to implement the system, test it with real scenarios, and deploy a working version of Nubl.

10. References

[1] Richey, M., Brooks, J. W., & Ravishankar, M. N. (2024). Leveraging digital tools to foster resilience in the charity sector: The case of eBay's Charity Connect initiative. *Information Systems Journal*, 35(2), 646–673.
https://doi.org/10.1111/isj.12554

[2] Fajardo-Steinhäuser, M. (2024, January 30). *Digital payments for humanitarian aid in Pakistan*. International Growth Centre.
https://www.theigc.org/blogs/digital-payments-humanitarian-aid-pakistan

[3] Callen, M., Fajardo-Steinhäuser, M., Findley, M., & Ghani, T. (2025). *Can digital aid deliver during humanitarian crises?* CESifo Working Paper No. 113670.
https://doi.org/10.1287/mnsc.2024.06469

[4] Carboni, A., Russo, D., Moroni, D., & Barsocchi, P. (2023). Privacy-by-design in systems for assisted living, personalised care, and wellbeing: A stakeholder analysis. *Frontiers in Digital Health*, 4, Article 934609.
https://doi.org/10.3389/fdgth.2022.934609

[5] Kumar, P., Ozawa, S., Tiwari, A., & Xu, H. (2024). Distributed ledger–based health credential verification in cross-border settings. *Management Science* (Advance online publication).
https://doi.org/10.1287/mnsc.2024.06469

[6] Murthy, V. H., Ghassemi, M., & Kohane, I. S. (2023). Bridging the digital health divide: Ensuring equitable access to digital health technologies. *Digital Medicine*, 6(1), 1–8.
https://pmc.ncbi.nlm.nih.gov/articles/PMC10648726/

[7] International Organization for Standardization. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (2nd ed.).
https://www.iso.org/standard/78176.html

[8] International Organization for Standardization. (2018). *ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering*.
https://www.iso.org/standard/72089.html

[9] European Union. (2016). *General Data Protection Regulation (GDPR) (EU) 2016/679*.
https://eur-lex.europa.eu/eli/reg/2016/679/oj

[10] National Cybersecurity Authority (NCA), Kingdom of Saudi Arabia. (2021). *Personal Data Protection Law (PDPL)*. A7 الممكّنات النظامية

[11] United Nations. (2015). *Sustainable Development Goals* (SDG 2 – Zero Hunger; SDG 9 – Industry, Innovation, and Infrastructure).
https://sdgs.un.org/goals

[12] Association for Computing Machinery (ACM) & IEEE Computer Society. (2018). *Software Engineering Code of Ethics and Professional Practice* (Version 5.2).
https://ethics.acm.org/code-of-ethics

[13] *Technical Reference & Functional Requirement Modularity Based on Linked Modules in the Main Documentation*. Technical Reference & FR Modularity Based on the Modulus Linked to Functional Requirement in the Main Documentation .docx
[14] Barakah App. *Barakah – fresh food, saved*.
https://www.barakah.app/
[15] Saudi Data & AI Authority (SDAIA). *Ehsan platform*.
https://ehsan.sa/
[16] World Food Programme (WFP). *WFP SCOPE: Beneficiary information and transfer management platform*.
https://usermanual.scope.wfp.org/
[17] RedRose. *RedRose – ONEsolution for the humanitarian sector*.
https://www.redrose.io/
[18] United Nations High Commissioner for Refugees (UNHCR). *Cash-based interventions*.
https://www.unhcr.org/what-we-do/respond-emergencies/cash-based-interventions
[19] Bonat. *Bonat – customer engagement and loyalty solution*.
https://www.bonat.io/en/home
[20] MyFatoorah. *MyFatoorah API V2 documentation*.
https://docs.myfatoorah.com/docs/overview
[21] National Cybersecurity Authority (NCA). *Privacy statement*.
https://nca.gov.sa/en/privacy-statement/
[22] European Data Protection Board (EDPB). *Guidelines, recommendations and best practices*.
https://www.edpb.europa.eu/our-work-tools/general-guidance/guidelines-recommendations-best-practices_en
[23] Al Bir Charity Association in Medina. *Official website*.
https://biralmadinah.org.sa/

Appendices

Appendix A: Teamwork and Project Management Evidence

A.1 Teamwork Tools and Collaboration

Our team used these tools during Capstone I:

- Jira for tasks, assignments, and progress tracking
- Teams / WhatsApp for communication and quick updates
- Word for report writing and final formatting
- Diagram tools (Drow.io / Lucid chart and others) for UML and ERD deliverables

Work was divided between team members. Each major output was reviewed by the team before adding it to the final report.

A.2 Meetings and Communication Log

Supervisor Meetings

(Information below is taken from the official supervision record forms you provided.)

#	*Date*	*Type*	*Attendees*	*Main Outcome*
1	29/09/2025	Supervisor meeting	Salem Shurrab, Bashar Naial, Osamah Al-Suraimi, Dr. Basit Shahzad	First introductory meeting. Discussed initial capstone ideas and general expectations.
2	05/10/2025	Supervisor meeting	Salem Shurrab, Bashar Naial, Osamah Al-Suraimi, Dr. Basit Shahzad	Discussed team coordination, competitor analysis, Jira setup, and general project concept.
3	12/10/2025	Supervisor meeting	Salem Shurrab, Bashar Naial, Osamah Al-Suraimi, Dr. Basit Shahzad	Reviewed Phase 1 completion, project naming, and discussed payment options and next phase direction.
4	12/10/2025	Supervisor meeting	Salem Shurrab, Bashar Naial, Osamah Al-Suraimi, Dr. Basit Shahzad	Reviewed functional and non-functional requirements and discussed correct requirement sources.
5	19/10/2025	Supervisor meeting	Salem Shurrab, Bashar Naial, Osamah Al-Suraimi, Dr. Basit Shahzad	Gave feedback on FR/NFR, discussed SLR tools, and planned meetings with real stakeholders.
6	24/10/2025	Supervisor meeting	Salem Shurrab, Bashar Naial, Dr. Basit Shahzad	Reviewed stakeholder updates, evaluated architecture options, and set a deadline for detailed design completion.
7	07/12/2025	Supervisor meeting	Salem Shurrab, Bashar Naial, Osamah Al-Suraimi, Dr. Basit Shahzad	Trying to address communication issues, design disagreements, and planned the final report, diagrams, and presentation.

8	14/12/2025	Supervisor meeting	Salem Shurrab, Bashar Naial, Dr. Basit Shahzad	Reviewed compliance, refined the report, addressed feedback, and focused on finalizing the report.

Table 11: Supervisor Meetings

Stakeholder Meetings

#	*Date*	*Type*	*Attendees*	*Main Outcome*
1	28/10/2025	Stakeholder interview	Project team + local stakeholders	Collected initial real-world requirements and constraints.
3	06/11/2025	Stakeholder Interview	Project team + local stakeholders (Sponsor)	Clarified the Nubl project vision and its differentiation from existing charity platforms confirmed the project's uniqueness and social impact potential.
3	08/11/2025	Stakeholder interview	Project team + stakeholders	Validated requirements and clarified operational workflow.
4	08/12/2025	Stakeholder interview	Project team + stakeholders	Final confirmation of requirements before Capstone I submission.

Table 12: Stakeholder Meetings

Team Sync Meetings

Regular internal team meetings were held **twice every week** to ensure progress and coordination.

Period	*Days*	*Time*	*Duration*	*Attendees*	*Purpose*
16 Sep – 22 Dec 2025 (14 weeks)	Every **Sunday & Thursday**	**1:00 – 2:00 PM**	1 hour	Salem Shurrab, Bashar Naial, Osamah Al-Suraimi	Task review, diagram discussion, Jira updates, and report coordination

Table 13: Team Sync Meetings

A.3 Jira Task Tracking Evidence

The table below is an extract from our Jira export. It shows real tasks, owners, and status. *(Full Jira export is available as evidence if needed.)*

Issue Key	*Summary*	*Status*	*Assignee*	*Created*	*Updated*
ECS-40	Create Component Diagram	Done	Bashar Naial	Dec 17, 2025, 7:02 PM	Dec 17, 2025, 11:01 PM
ECS-31	Holding informal consultative meetings	Done	Bashar Naial	Dec 04, 2025, 6:45 PM	Dec 04, 2025, 6:45 PM
ECS-29	Conduct a stakeholder interview	Done	Bashar Naial	Dec 04, 2025, 6:41 PM	Dec 04, 2025, 6:41 PM
ECS-39	Architecture Options Evaluation & Justification	Done	OsamX	Dec 14, 2025, 12:52 AM	Dec 14, 2025, 1:18 AM
ECS-37	Design Entity Relationship Diagram	Done	OsamX	Dec 14, 2025, 12:10 AM	Dec 14, 2025, 12:12 AM
ECS-36	Prepare UML diagrams that represent the system behavior for core FRs	Done	OsamX	Dec 13, 2025, 11:59 PM	Dec 14, 2025, 12:12 AM

ECS-33	Report Finalization	In Progress	Salem Shurrab	12/11/2025 16:05	12/11/2025 16:06
ECS-20	Add WBS Plan	Done	Salem Shurrab	12/4/2025 18:34	12/11/2025 16:05

Table 14: Jira Task Tracking Evidence

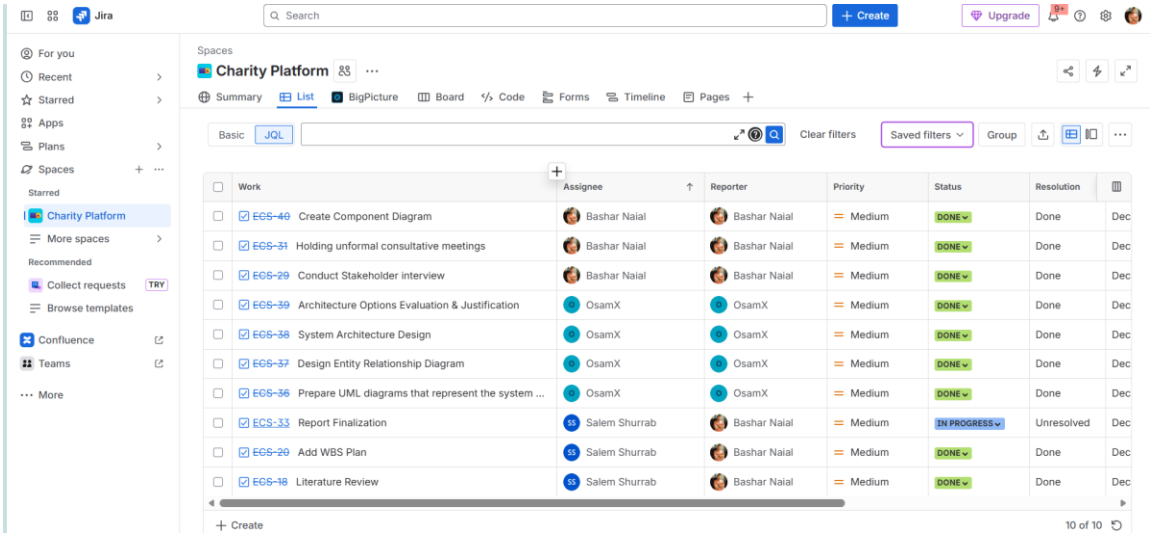


Figure 25 Jira Tasks

A.4

Risk Register

Risk ID	*Risk*	*Probability*	*Impact*	*Mitigation*	*Owner*	*Status*
R-01	Provider onboarding may be slow	Medium	High	Simple onboarding steps + short training	Salem	Managed
R-02	Privacy risk (PII exposure)	Low	High	RBAC + minimal data + audit logs	Osamah	Managed
R-03	Payment integration issues	Medium	Medium	Use MyFatoorah only + test with sandbox	Bashar	Planned

Table 15: Risk Register

Appendix B: Proof of Concept Evidence

To support our design, we prepared a proof of concept (PoC). This PoC is used to validate the main user journey and the core system workflow.

PoC Type: [Running web prototype / Clickable UI prototype]

PoC Link: [NUBL - Unified Prototype] this is for the beneficiaries interface

Repository : [shglp1/UserDemoInterface]

Demo video : [Demo.mp4]

B.1 What the PoC shows

- Role-based access: each role (Beneficiary / Donor / Provider / Admin) but now the link is for the Beneficiary
- Main workflow: request → admin review → approval → QR redemption → receipt upload
- Donor privacy: donor sees totals and statistics only (no beneficiary identity)

B.2 Figures to include (screenshots)

- Figure B-1: Login / role access

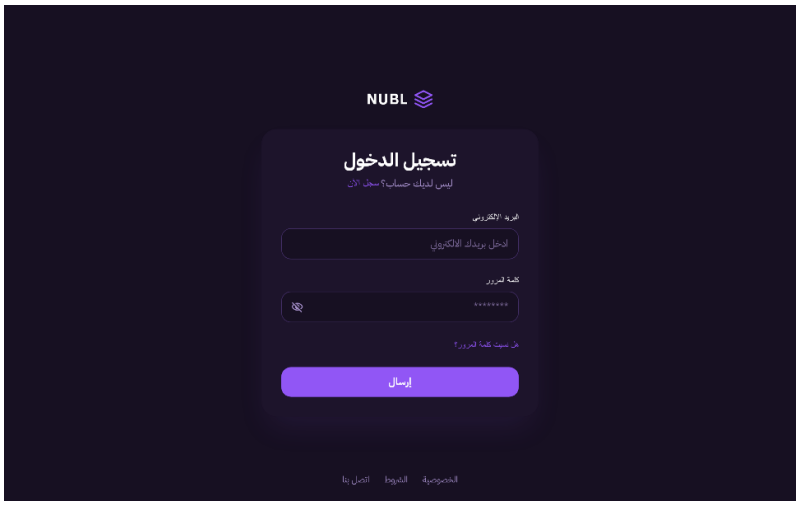


Figure 26: UI UX Login / role access

- Figure B-2: Beneficiary request creation

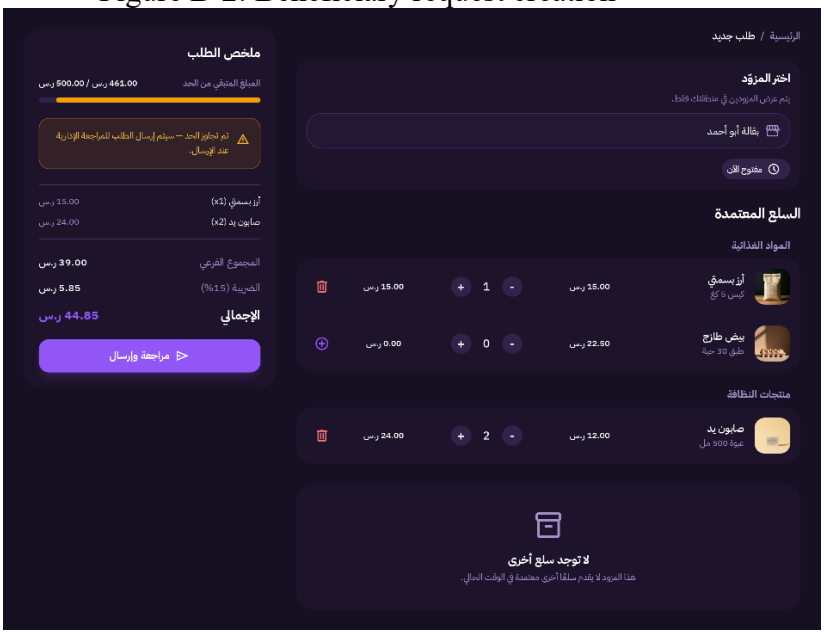


Figure 27: UI UX Beneficiary request creation

- Figure B-3: Admin approval screen

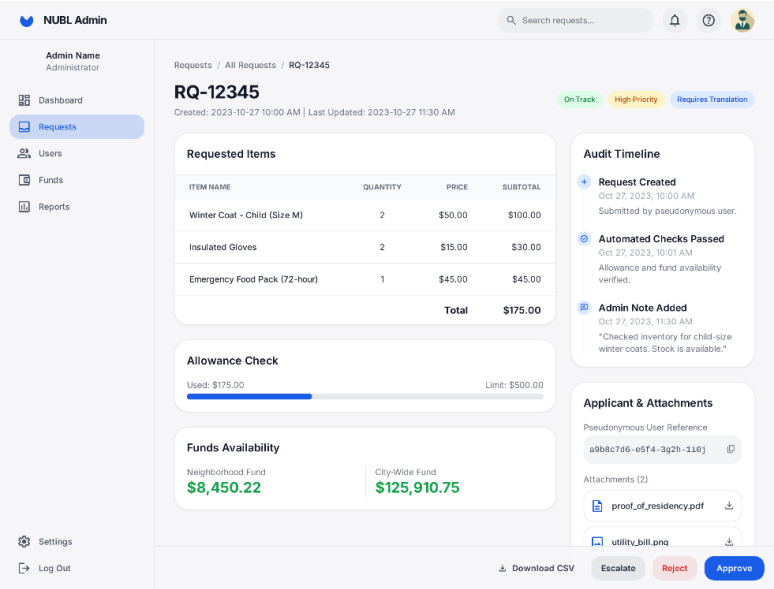


Figure 28: UI UX Admin approval screen

- Figure B-4: Provider QR redemption + receipt upload

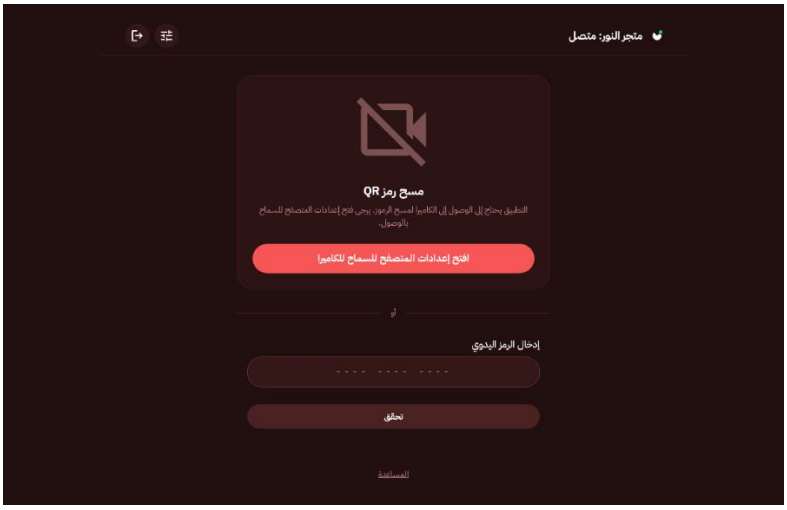


Figure 29: UI UX Provider QR redemption + receipt upload

- Figure B-5: Donor dashboard (aggregated)

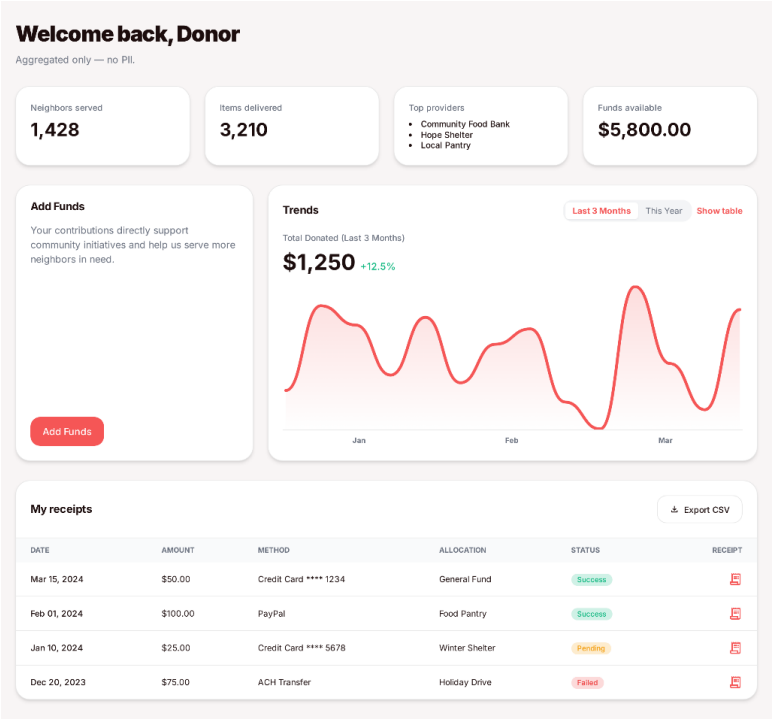


Figure 30: UI UX Donor dashboard (aggregated)

Appendix C: Continuous Development

C.1 New Knowledge We Needed

During Capstone I, we needed to learn topics that support our design and requirements work, such as payment workflow, QR redemption flow, requirements writing standards, and security/privacy basics.

C.2 Resources We Used

- MyFatoorah documentation (payment process)
- OWASP guidance (security basics)
- ISO/IEC/IEEE 29148 (requirements structure)
- ISO/IEC 25010 (quality model for NFRs)
- Saudi privacy requirements and privacy-by-design principles

C.3 How We Used What We Learned

Topic	*Resource*	*How we applied it in the project*
Requirements writing	ISO 29148	We wrote FRs/NFRs in a clear structured way.
NFR quality	ISO 25010	We linked NFRs to quality attributes like security and reliability.
Security & privacy	OWASP + privacy principles	We planned RBAC, limited access, and included audit logging in design.

Payment workflow	MyFatoorah docs	We designed the donation flow using one gateway in the MVP.
QR redemption	Design workflow + diagrams	We designed provider redemption using one-time QR and receipt proof.

Table 16: What We Learned

Appendix D: Logs Table

Date	Author	Version	Change Reference
September 20, 2025	Salem Hussameldeen Shurrab	1.0	Initial document creation. Defined problem statement, motivation, and early idea for a neighborhood-based food assistance system.
September 23, 2025	Salem Hussameldeen Shurrab	1.1	Added background section and clarified the social problem related to dignity, privacy, and aid distribution.
September 26, 2025	Project Team	1.2	First supervisor discussion. Refined project direction and confirmed feasibility as Capstone I (analysis & design).
October 2, 2025	Project Team	1.3	Added initial objectives and early scope definition focusing on MVP.
October 7, 2025	Project Team	1.4	Conducted early stakeholder outreach and aligned project idea with real charity workflows.
October 12, 2025	Project Team	1.5	Updated objectives to explicitly target MVP outcomes and clarified system purpose beyond implementation only.
October 18, 2025	Project Team	2.0	Introduced structured SRS sections, initial use case diagram, and role definitions (Donor, Beneficiary, Provider, Admin).
October 24, 2025	Project Team	2.2	Added literature review focusing on digital aid platforms, privacy-by-design, and humanitarian systems.
October 30, 2025	Project Team	2.4	Expanded ethics, legal considerations, and privacy requirements aligned with PDPL and GDPR.
November 4, 2025	Project Team	2.6	Added ERD draft and mapped core entities to system workflows.
November 9, 2025	Project Team	3.0	Major restructuring based on rubric review. Improved document consistency, formatting, and section alignment.
November 13, 2025	Project Team	3.2	Added SDLC methodologies comparison and justified Waterfall model selection for Capstone context.
November 18, 2025	Project Team	3.3	Refined project scope with explicit in-scope and out-of-scope subsections.

November 24, 2025	Project Team	3.4	Expanded constraints section to include technical, operational, security, and regulatory constraints.
November 29, 2025	Project Team	3.5	Reworked functional requirements to be actor-based and aligned with IEEE SRS practices.
December 4, 2025	Project Team	3.6	Categorized non-functional requirements by type (performance, security, usability, scalability, etc.).
December 10, 2025	Project Team	3.6.1	Added detailed architecture component responsibilities and explained connections among components.
December 15, 2025	Project Team	3.6.1	Verified ERD consistency against functional requirements and ensured all entities are properly related.
December 18, 2025	Project Team	3.6.1	Final audit pass: reference formatting, language polishing, rubric coverage verification, and submission readiness.

Table 17: Logs Table